

Ionic Angular oefenapp

WEBAPP

Verone de Vries

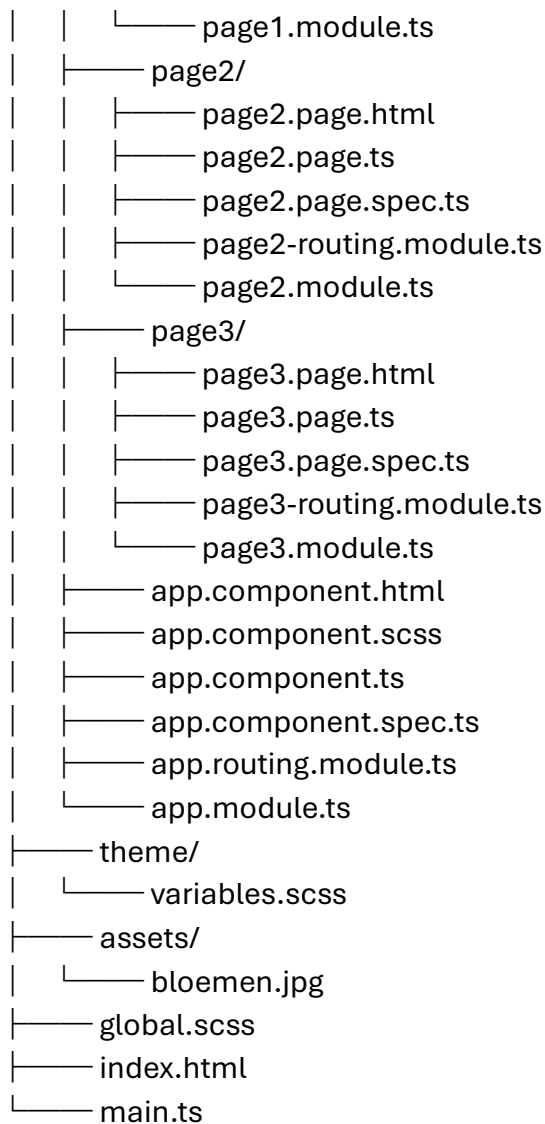
juli 2026

Inhoud

1.	Mappenstructuur	2
2.	Userflow en Guard	4
3.	Fasen	11
4.	APP	15
5.	WELCOME.....	19
6.	SIGNUP	23
7.	LOGIN	30
	7.1. Loginservice	44
8.	Homepage	47
9.	PAGES	54
	9.1. HOME.....	54
	9.2. PAGE1	59
	9.3. PAGE2	63
	9.4. PAGE3	67
10.	Global en Variables SCSS	71
11.	INDEX.HTML	77
12.	MAIN.TS	78
13.	Firebase authentication en databases	79
	13.1. Live Google Firebase backend	79

1. Mappenstructuur

```
src/
├── app/
│   ├── guards/
│   │   ├── login.guard.ts
│   │   └── login.guard.spec.ts
│   ├── services/
│   │   ├── login.service.ts
│   │   ├── login.service.spec.ts
│   │   └── signup.ts
│   ├── welcome/
│   │   ├── welcome.page.html
│   │   ├── welcome.page.scss
│   │   ├── welcome.page.ts
│   │   ├── welcome-routing.module.ts
│   │   └── welcome.module.ts
│   ├── login/
│   │   ├── login.page.html
│   │   ├── login.page.scss
│   │   ├── login.page.ts
│   │   ├── login.page.spec.ts
│   │   ├── login.routing.module.ts
│   │   └── login.module.ts
│   ├── homepage/
│   │   ├── homepage.page.html
│   │   ├── homepage.page.scss
│   │   ├── homepage.page.ts
│   │   ├── homepage.page.spec.ts
│   │   ├── homepage-routing.module.ts
│   │   └── homepage.module.ts
│   ├── home/
│   │   ├── home.page.html
│   │   ├── home.page.scss
│   │   ├── home.page.ts
│   │   ├── home-routing.module.ts
│   │   └── home.module.ts
│   └── page1/
│       ├── page1.page.html
│       ├── page1.page.scss
│       ├── page1.page.ts
│       ├── page1.page.spec.ts
│       └── page1-routing.module.ts
```



De signOut functionaliteit (het uitloggen uit de Firebase Cloud) zit gecentraliseerd op **twee specifieke plekken** in jouw mappenstructuur:

1. De Database Actie

De daadwerkelijke, technische Firebase-koppeling die de sessietoken in de cloud vernietigt, bevindt zich in:

- **src/app/services/login.service.ts**

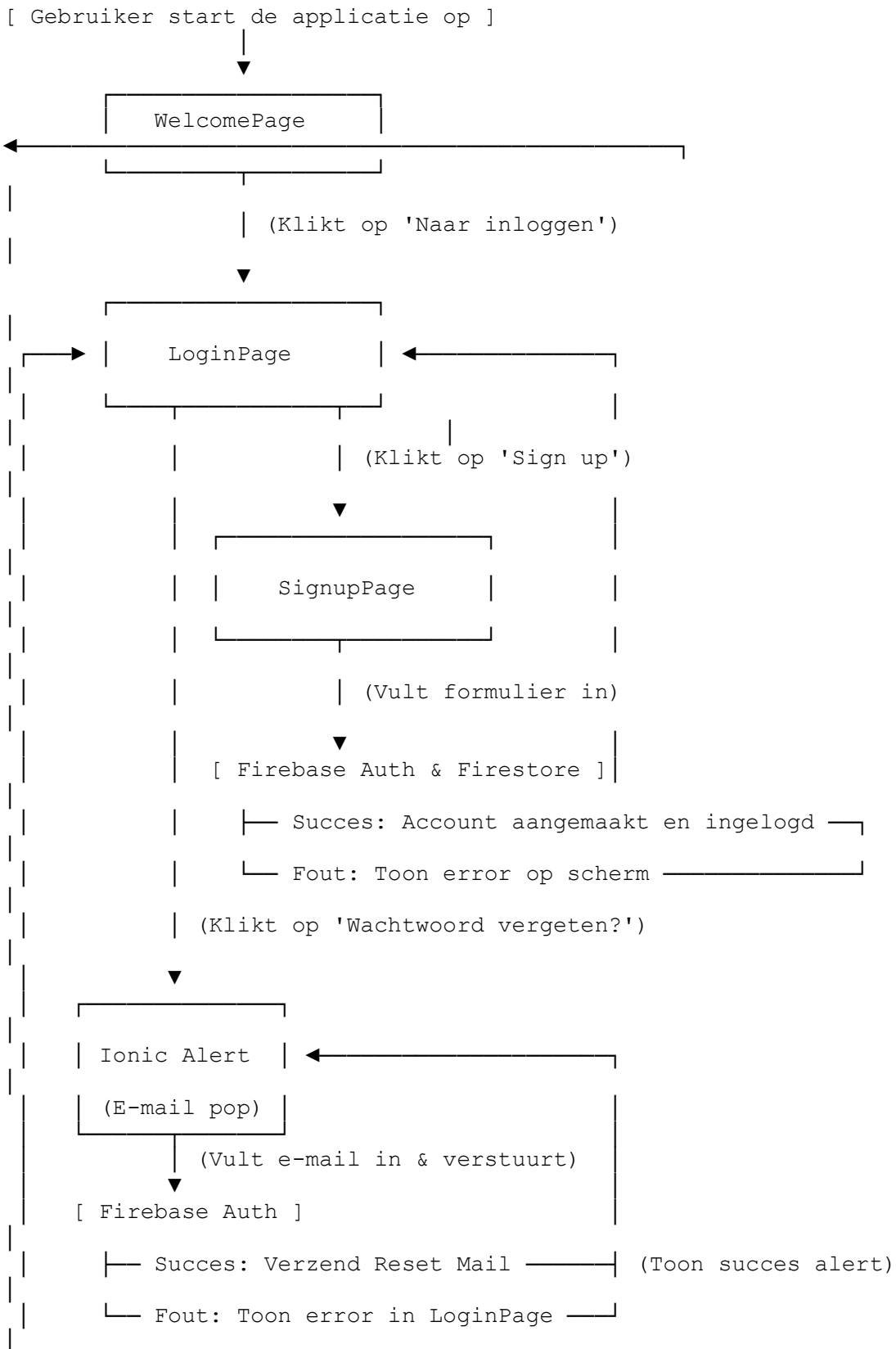
2. De Gebruikers Actie

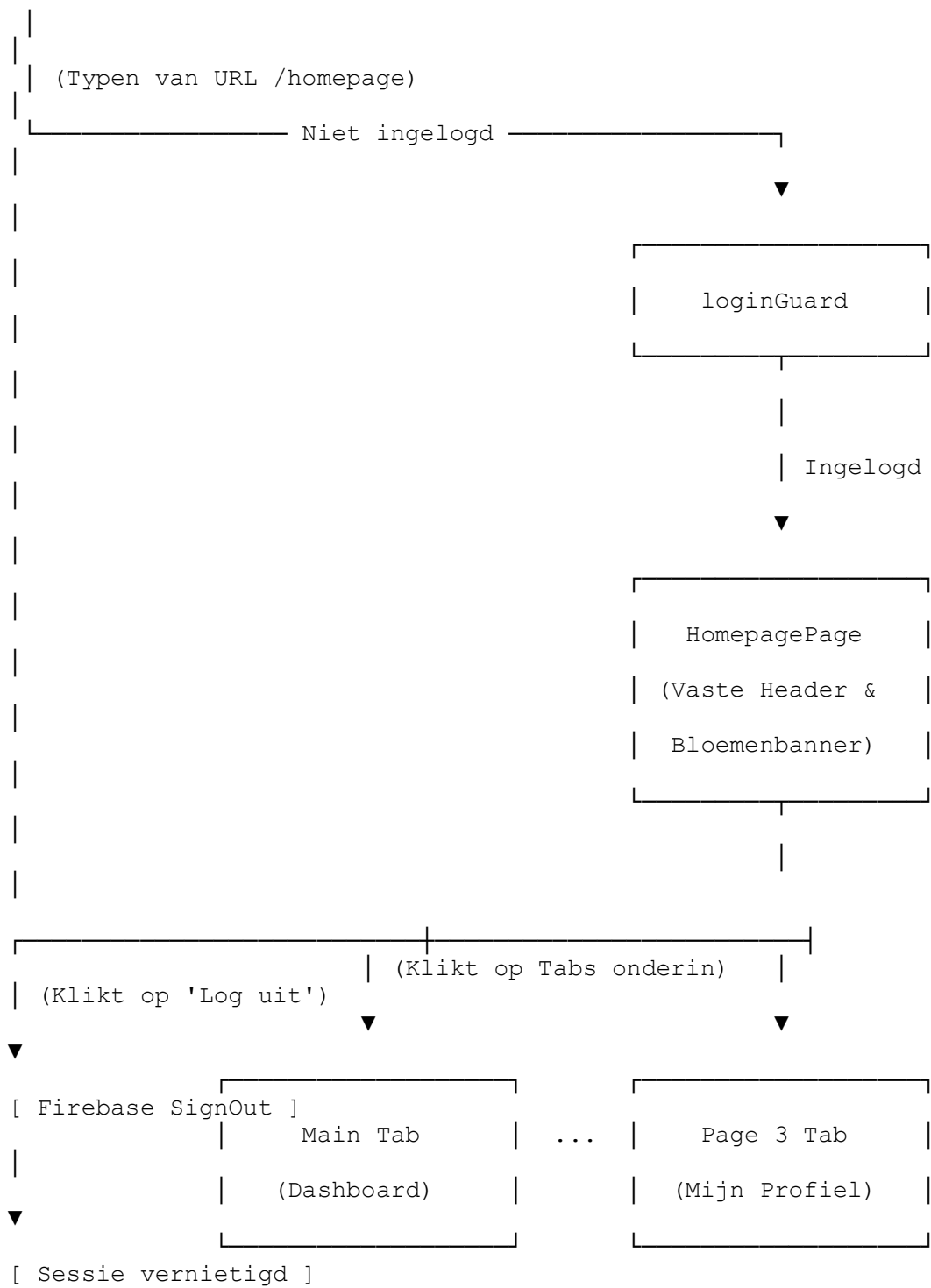
De logica die reageert op de klik van de gebruiker en daarna de routing aanstuurt, bevindt zich in:

- **src/app/homepage/homepage.page.ts**

2. Userflow en Guard

Dit schema laat exact zien hoe een gebruiker zich door de app beweegt en hoe de frontend (Ionic) communiceert met de backend (Firebase Auth & Firestore).





Toelichting per fase van de stroom

1. De Opstartfase (Publiek)

- De gebruiker komt binnen op de WelcomePage (het basispad /, dat automatisch redirect naar /welcome).
- Vanuit hier kan de gebruiker via de centrale knop navigeren naar de LoginPage.
- Vanaf de inlogpagina kan de gebruiker doorklikken naar de SignupPage om een account aan te maken.

2. Het Wachtwoordherstelproces (On-demand Security)

- Wanneer de gebruiker op de LoginPage op "Wachtwoord vergeten?" klikt, opent er een interactieve Ionic Alert (pop-up) op hetzelfde scherm.
- De gebruiker vult zijn e-mailadres in binnen de pop-up en klikt op verzenden.
- De frontend doet een asynchrone oproep naar de LoginService, die via Firebase Authentication (sendPasswordResetEmail) een unieke en beveiligde herstellink genereert.
- **Bij succes:** Krijgt de gebruiker een bevestiging via een succes-toast of alert en verstuurt Firebase automatisch de e-mail. De gebruiker blijft op de LoginPage.
- **Bij een fout:** Vangt de app de Firebase-foutcode op (zoals een onbekend e-mailadres) en toont deze direct in een duidelijke fout-alert pop-up.

3. Het Registratieproces (De 2-Steps Backend Brug)

Wanneer de gebruiker op "Registreren" klikt, start de Signup-service een dubbele actie in de cloud via Firebase:

1. **Firebase Authentication** maakt de unieke gebruiker (UID) aan met e-mail en wachtwoord.
2. **Cloud Firestore** slaat direct daarna de aanvullende gegevens (zoals voornaam en geboortedatum) op in de users-collectie onder dat exacte, corresponderende UID.
3. **Automatische Login:** Bij een succesvolle registratie logt Firebase de gebruiker automatisch in. De router stuurt de gebruiker direct door naar het beveiligde dashboard (/homepage), waardoor de handmatige inlogstap wordt overgeslagen.

4. De Poortwachter (Security)

- Zodra de app probeert te navigeren naar /homepage, springt de loginGuard (de poortwachter) direct aan.
- De guard controleert via een RxJS-pijplijn (userIsAuthenticated\$) live de authenticatiestatus bij Firebase met een veilige take(1) meting.
- **Niet ingelogd?** De guard blokkeert de route en kaapt de navigatie om de gebruiker direct terug te sturen naar de LoginPage.
- **Wel ingelogd?** De deur gaat open (return true) en de toegang tot het intranet wordt verleend.

5. Het Dashboard (Beveiligd)

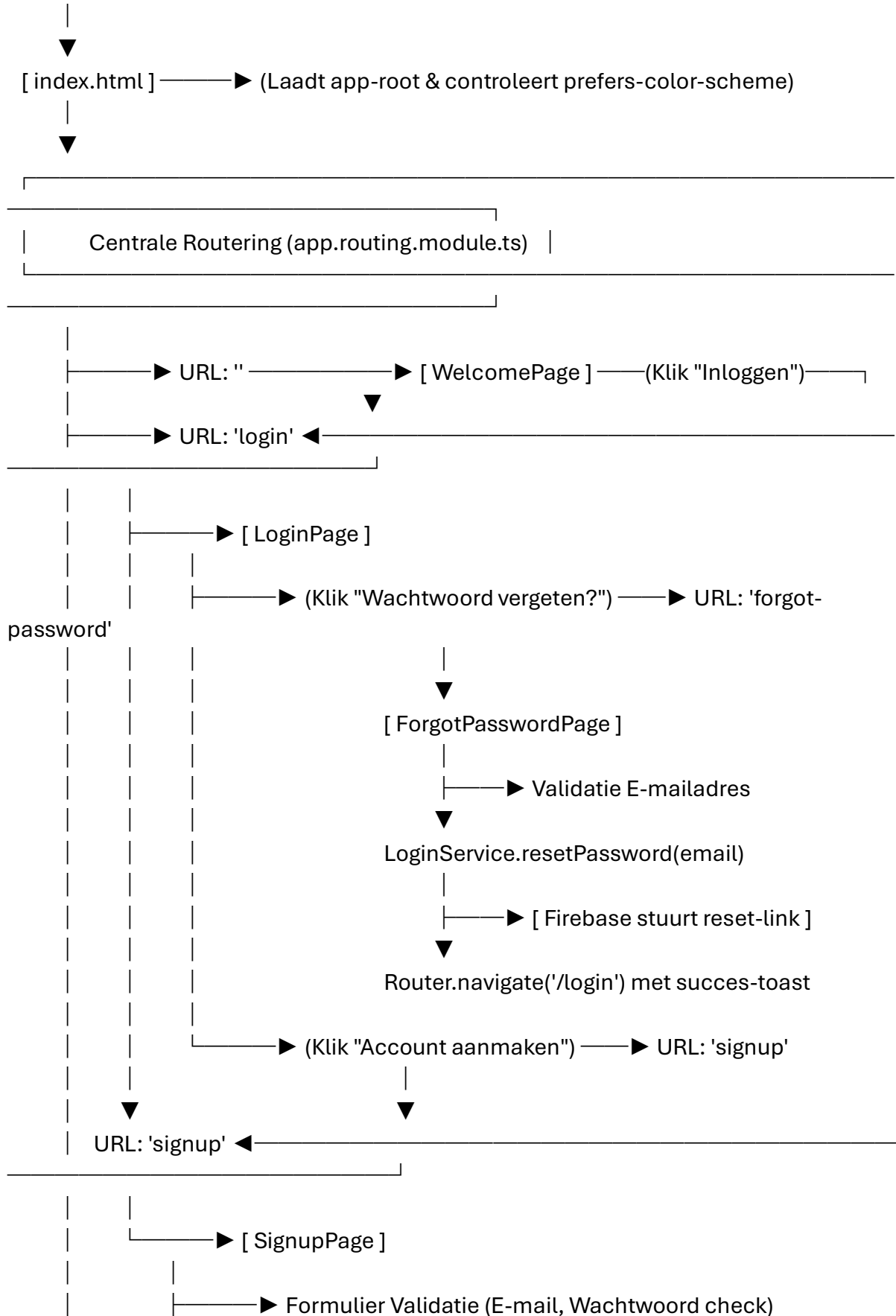
- De HomePagePage opent en fungeert als het vaste frame (de shell) van het intranet.
- De bovenbalk bevat een **dynamische titel** die live meeverandert met de actieve pagina, en een gecombineerde **Uitlogknop** (tekst + icoon) die de Firebase-sessie beëindigt en de LocalStorage opschooont.
- Binnen de <ion-tabs> structuur navigeert de gebruiker via de onderste menubalk soepel tussen de kinderroutes: main (Home), page1, page2 en page3, zonder dat de app telkens volledig opnieuw hoeft te laden.

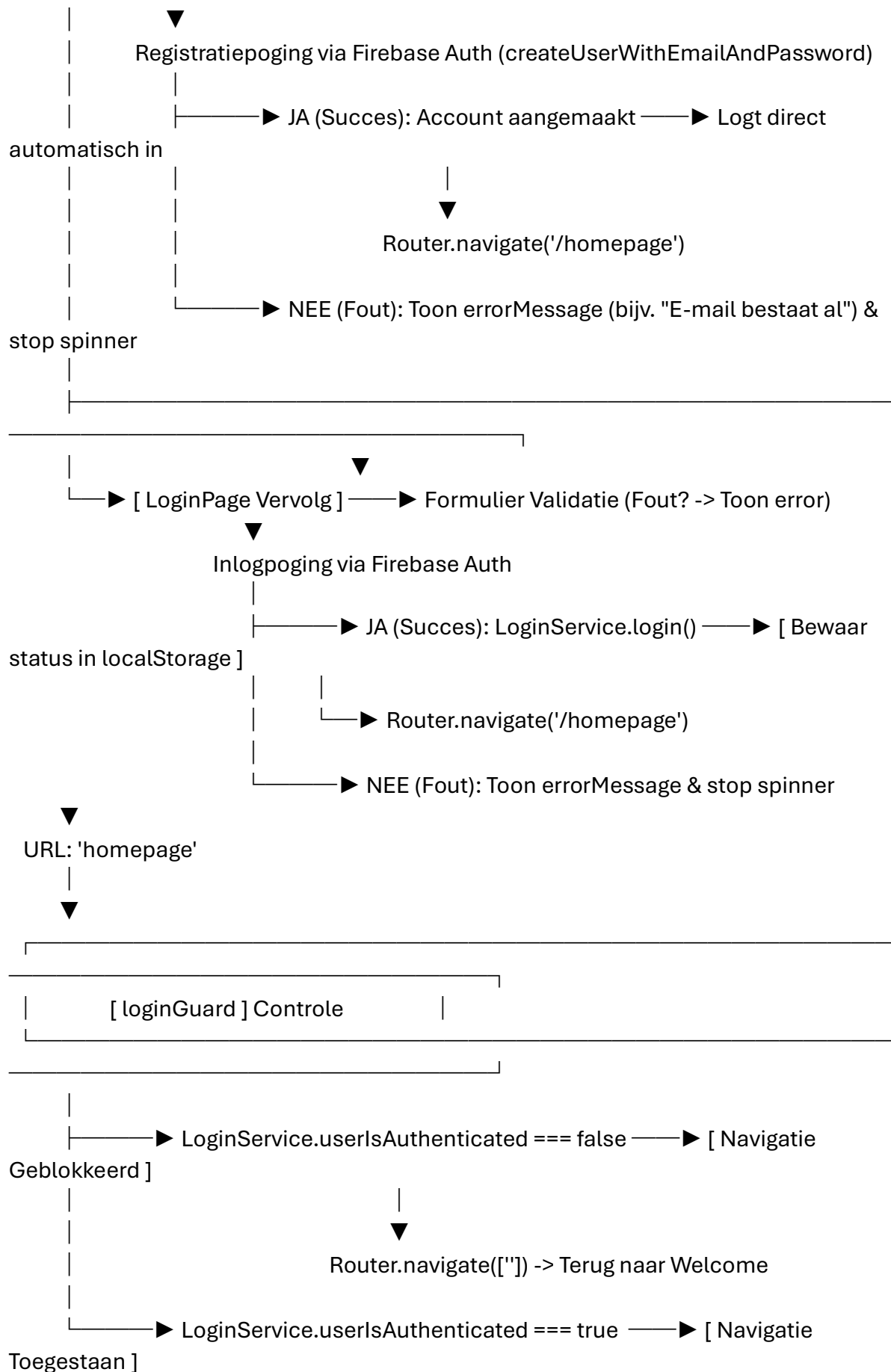
6. De Uitlogprocedure (Sessie Beëindiging)

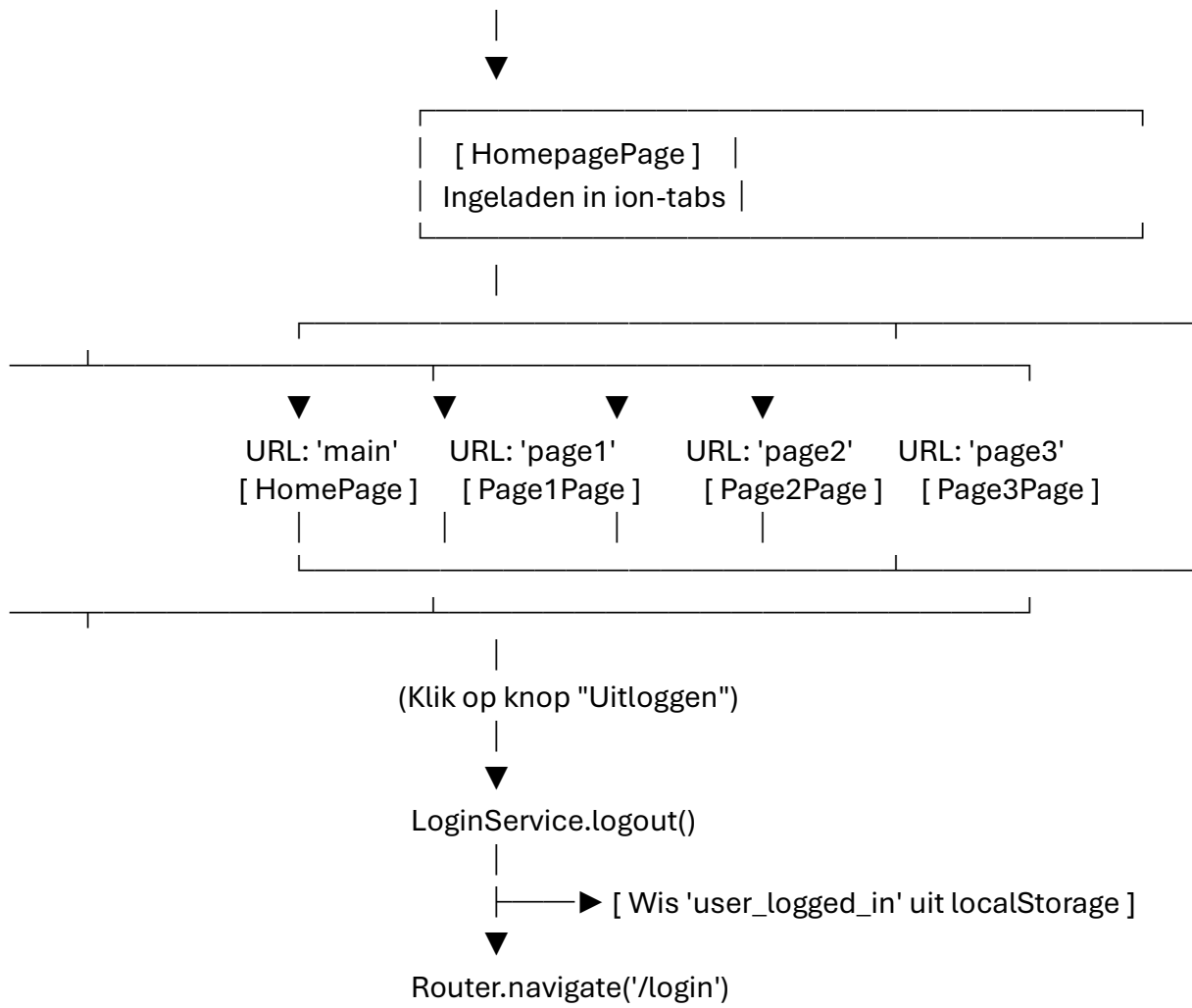
- Zodra de gebruiker op de uitlogknop klikt (in de header of de tab-balk), communiceert de HomePagePage via onLogout() asynchroon met Firebase (signOut()).
 - Het cloud-token wordt ongeldig gemaakt, de actieve sessie stopt en de eventuele back-up status in de LocalStorage wordt opgeschoond.
 - De router stuurt de gebruiker veilig terug naar de welkomspagina (/welcome) en overschrijft de navigatiegeschiedenis (replaceUrl: true) om ongeoorloofde terug-navigatie te blokkeren.
-

Gedetailleerd schema

[App Start: main.ts]







3. Fasen

Fase 1: De Opstartfase (Bootstrapping)

Dit gebeurt in de eerste milliseconden wanneer een gebruiker de app opent. Bestanden laden elkaar opvolgend in om de motor van de app te starten.

1. **tsconfig.json**

- **Functie:** De scheidsrechter vooraf. Voordat er überhaupt code naar de browser gaat, controleert dit bestand of alle TypeScript-code voldoet aan de strenge kwaliteits- en typeregels (strict: true). Het vertelt de compiler exact hoe de code vertaald moet worden naar modern, vlot leesbaar JavaScript.

2. **index.html**

- **Functie:** De funderingsplaat. Dit is het allereerste bestand dat de browser daadwerkelijk inlaadt. Het configureert de mobiele weergave (viewport-fit=cover), opent de beveiligingspoorten via de Content Security Policy (CSP) en biedt de lege <app-root> tag waarin de gehele applicatie zich later gaat nestelen.

3. **theme/variables.scss en global.scss**

- **Functie:** De schilder en de stukadoor. Voordat de componenten in beeld komen, laadt de browser deze stylesheets. variables.scss bepaalt de basiskleuren (zoals het specifieke bosgroen) en controleert of de telefoon op Dark Mode staat. global.scss past direct de universele scrolloverrides en lettertypes toe op de lege HTML-basis.

4. **main.ts**

- **Functie:** De startmotor. Dit script activeert Angular in de browser. Het voert de JIT-compiler uit en geeft de expliciete opdracht: "Bouw nu de AppModule op en start de applicatie". Als hier iets misgaat, crasht de app nog voordat er een pixel op het scherm verschijnt.

5. **app/app.module.ts**

- **Functie:** De centrale verdeelkast. Dit bestand verzamelt de absolute basisbenodigdheden van de app. Het start de core-functionaliteit van Ionic (IonicModule.forRoot()), initialiseert de Firebase-verbinding met de juiste cloud-configuratie en importeert de centrale routing.

6. **app/app.component.ts en app.component.html**

- **Functie:** Het chassis van de auto. Dit is de root-component (app-root). De HTML bevat enkel de <ion-router-outlet>. Dit bestand doet zelf visueel niets, maar fungeert als het lege frame waar de router straks alle dynamische pagina's in gaat schuiven.

7. **app/app-routing.module.ts**

- **Functie:** De verkeersregelaar. Dit bestand kijkt direct naar de URL in de browserbalk bij het opstarten. Omdat de app net opstart, is de URL leeg (""). De verkeersregelaar ziet dit en besluit op basis van de configuratie direct via een veilige redirect: "Stuur de gebruiker door naar /welcome en laad de WelcomePageModule in via lazy loading".

Fase 2: De Draai- en Gebruiksfase (Runtime)

De app is nu succesvol opgestart. Vanaf dit punt bepalen de acties van de gebruiker welke specifieke bestanden in actie komen.

8. **app/services/login.service.ts**

- **Functie:** De live-verbindingslijn met de cloud. Zodra de app in beweging komt, luistert deze service via de RxJS stream live naar Firebase Auth. Hij controleert geen lokale browser-opslag, maar vraagt direct aan de Google Firebase-servers of er een geldige, actieve sessie bestaat. Deze status (`userIsAuthenticated$`) stuurt de beveiliging van de rest van de app aan.

9. **app/welcome/welcome.module.ts (en bijbehorend -routing.module.ts)**

- **Functie:** De VIP-hostess. De module wordt door de router ingeladen via lazy loading en registreert de welkomspagina binnen de Angular-context.

10. **app/welcome/welcome.page.ts, .html en .scss**

- **Functie:** Het eerste welkomstscherf. De TypeScript-component controleert asynchroon de live Firebase-status via de LoginService. Is de gebruiker al ingelogd in de cloud? Dan wordt hij direct geruisloos doorgestuurd naar /homepage. Zo niet, dan toont de HTML de bloemenbanner en de knoppen om naar het inlog- of registratiescherf te gaan.

11. **app/login/login.module.ts (en bijbehorend .routing.module.ts)**

- **Functie:** De douanepost-module. Wordt ingeladen zodra de gebruiker naar /login navigeert. Deze module zorgt dat alle benodigde Ionic- en Angular-formulierfunctionaliteiten klaarstaan voor gebruik.

12. **app/login/login.page.ts, .html en .scss**

- **Functie:** Het inlogformulier en wachtwoordherstel.
 - *Inloggen:* De HTML vangt de invoer op en stuurt dit via `onSubmit()` veilig naar de LoginService. Deze controleert via Firebase Auth (`signInWithEmailAndPassword`) live in de cloud of de gegevens kloppen.
 - *Wachtwoord Vergeten:* Als de gebruiker op de link klikt, activeert de TS-component een `AlertController` (pop-up). Zodra de gebruiker zijn e-mail invult, activeert de service de Firebase-methode **`sendPasswordResetEmail()`**, waarna Firebase automatisch een officiële herstel-e-mail verstuurt.

13. `app/signup/signup.page.ts`, `.html` en `.scss`

- **Functie:** Het registratieformulier (De 2-staps backend brug). Wanneer een nieuwe medewerker zich registreert, voert de TypeScript-component een krachtige dubbele cloud-actie uit via de LoginService:

1. Hij maakt de authenticatie aan in Firebase Auth met e-mail en wachtwoord via `createUserWithEmailAndPassword`.

2. Bij succes gebruikt hij het gloednieuwe, unieke UID van de gebruiker om direct aanvullende gegevens (zoals voornaam en geboortedatum) op te slaan in de Cloud Firestore database onder de collectie users. Daarna volgt een automatische login en doorverwijzing naar `/homepage`.

14. `app/guards/login.guard.ts`

- **Functie:** De uitsmijter aan de deur. Voordat de router de gebruiker toelaat op `/homepage`, springt deze functionele guard naar voren. Hij maakt via RxJS (`take(1)`) een momentopname van de `userIsAuthenticated$` stream uit de LoginService. Staat het stoplicht in de Firebase-cloud op groen? Dan mag de router doorrijden. Staat het op rood (niet ingelogd)? Dan kaapt de guard de route en flitst hij de gebruiker direct terug naar de LoginPage.

15. `app/homepage/homepage.module.ts` en `homepage-routing.module.ts`

- **Functie:** De dashboard-architect. Laadt de hoofdcontainer van het intranet in en configureert de vier onderliggende sub-routes (tabs): `main`, `page1`, `page2` en `page3`.

16. `app/homepage/homepage.page.ts`, `.html` en `.scss`

- **Functie:** De menustructuur (Tabs) & Uitlogknop. De HTML toont de bovenbalk met de **dynamische titel** (die live meeverandert met de actieve pagina) en de onderste navigatiebalk met de iconen. Wanneer de gebruiker op de uitlogknop klikt, triggert de TS-component de `onLogout()` methode. Deze roept `logout()` aan in de LoginService, waardoor Firebase (`signOut()`) de token vernietigt. De sessie stopt direct in de cloud en de router stuurt de gebruiker veilig terug naar de welkomstpagina (**/welcome**) met `replaceUrl: true` om terugklikken te blokkeren.

17. `app/home/`, `app/page1/`, `app/page2/`, `app/page3/`

- **Functie:** De werkvloer (Content). Afhankelijk van op welk tabblad de gebruiker klikt, laadt de router de specifieke kind-module in binnen de `<ion-tabs>`. De HTML toont de content (zoals de matrix-grids van het intranet). De TS-componenten kunnen live gegevens ophalen uit Firestore (zoals de voornaam van de ingelogde medewerker) om de pagina's direct te personaliseren.

Waar dienen de .spec.ts bestanden voor?

Bestanden zoals `login.service.spec.ts` of `page1.page.spec.ts` draaien nooit mee in de productie-app die de eindgebruiker te zien krijgt. Zij hebben een controlerende functie tijdens het ontwikkelen. Wanneer de ontwikkelaar het commando `npm run test` typt, bootsen deze bestanden een geïsoleerde mini-applicatie na om via unit-tests automatisch te controleren of alle logica, Firebase-koppelingen en formuliervalidaties nog perfect, stabiel en foutloos werken.

4. APP

1. app.module.ts (De Hoofdgereedschapskist / De Motor)

- **Functie:** Dit bestand start de complete applicatie op en verzamelt alle wereldwijde bibliotheken.
- **Wat het doet:** Dit is de belangrijkste module van je project. Het laadt de kernonderdelen van Angular en Ionic in en zet de officiële cloudverbinding op met jouw Firebase-project (Authentication en Firestore). Het vertelt de computer welk gereedschap overal in de app gebruikt mag worden en start de app op via de AppComponent.

2. app.routing.module.ts (De Hoofdverkeersleider)

- **Functie:** Dit bestand bepaalt de hoofdroutes (de URL-topstructuur) van je applicatie.
- **Wat het doet:** Dit bestand deelt de app op in grote, logische wegen (/ , /login, /signup, /homepage). Het regelt *lazy loading*, zodat pagina's pas worden ingeladen als de gebruiker erop klikt. Bovendien hangt dit bestand de loginGuard (de poortwachter) als een slot op de deur van de /homepage, waardoor de hele achterliggende applicatie in één klap is beveiligd tegen onbevoegde bezoekers.

3. app.component.html (De Hoofd-Blauwdruk / Het Frame)

- **Functie:** Dit is de allereerste HTML die wordt ingeladen en fungeert als de container voor de hele app.
- **Wat het doet:** Dit bestand bevat de <ion-app> tag die het Ionic-framework voor je activeert (voor mobiele animaties en styling). Het belangrijkste onderdeel hierin is de <ion-router-outlet>. Dit is een dynamisch whiteboard waarop de Hoofdverkeersleider (app.routing.module.ts) continu de juiste pagina plakt (eerst WelcomePage, daarna LoginPage, en na het inloggen HomepagePage).

Samen vormen deze bestanden het fundament waar de rest van jouw intranet op rust.

app.component.html

```
<ion-app>
  <ion-router-outlet></ion-router-outlet>
</ion-app>
```

app.component.spec.ts

```
import { CUSTOM_ELEMENTS_SCHEMA } from '@angular/core';
import { TestBed } from '@angular/core/testing';

import { AppComponent } from './app.component';

describe('AppComponent', () => {

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [AppComponent],
      schemas: [CUSTOM_ELEMENTS_SCHEMA],
    }).compileComponents();
  });

  it('should create the app', () => {
    const fixture = TestBed.createComponent(AppComponent);
    const app = fixture.componentInstance;
    expect(app).toBeTruthy();
  });

});
```

app.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: 'app.component.html',
  styleUrls: ['app.component.scss'],
  standalone: false,
})
export class AppComponent {
  constructor() {}
}
```

app.module.ts

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { RouteReuseStrategy } from '@angular/router';

import { IonicModule, IonicRouteStrategy } from '@ionic/angular';

import { AppComponent } from './app.component';
import { AppRoutingModuleModule } from './app.routing.module';

// De hypermoderne AngularFire 20 imports
import { provideFirebaseApp, initializeApp } from '@angular/fire/app';
import { provideFirestore, getFirestore } from '@angular/fire/firestore';
import { provideAuth, getAuth } from '@angular/fire/auth';
import { environment } from '../environments/environment';

@NgModule({
  declarations: [AppComponent],
  imports: [
    BrowserModule,
    IonicModule.forRoot(),
    AppRoutingModuleModule
  ],
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },

    provideFirebaseApp(() => initializeApp(environment.firebaseConfig)),
    provideFirestore(() => getFirestore()),
    provideAuth(() => getAuth())
  ],
  bootstrap: [AppComponent],
})
export class AppModule {}
```

app.routing.module.ts

```
import { NgModule } from '@angular/core';
import { PreloadAllModules, RouterModule, Routes } from '@angular/router';
import { loginGuard } from './login/login.guard';

const routes: Routes = [
  { // 1. BIJ OPSTARTEN: Stuur de gebruiker direct door naar /welcome
    path: '',
    redirectTo: 'welcome',
    pathMatch: 'full'
  },
  { // 2. DE WELKOMSTPAGINA: Nu op zijn eigen stabiele route
    path: 'welcome',
    loadChildren: () => import('./welcome/welcome.module').then(m =>
m.WelcomePageModule)
```

```

    },
    { // Route naar login-pagina
      path: 'login',
      loadChildren: () => import('./login/login.module').then(m =>
m.LoginPageModule)
    },
    { // Route naar signup-pagina
      path: 'signup',
      loadChildren: () => import('./signup/signup.module').then(m =>
m.SignupPageModule)
    },
    { // Homepage met navbalk en router-outlet beveiligd door guard
      path: 'homepage',
      loadChildren: () => import('./homepage/homepage.module').then(m =>
m.HomePageModule),
      canActivate: [loginGuard]
    }
  ];

@NgModule({
  imports: [
    RouterModule.forRoot(routes, { preloadingStrategy: PreloadAllModules })
  ],
  exports: [RouterModule]
})
export class AppRoutingModule { }

```

5. WELCOME

Dit is de specifieke functie van elk bestand in de map welcome:

1. `welcome.page.html` (De Blauwdruk)

- **Functie:** Dit bepaalt de structuur en de inhoud van het welkomstscherm.
- **Wat het doet:** Hierin staan de teksten (zoals "*Welkom op onze bedrijfspagina*"), de bloemenafbeelding bovenaan en de knop "*Naar Inloggen*". Dankzij de code `routerLink="/login"` op de knop, stuurt dit bestand de gebruiker bij een klik direct door naar het inlogscherm, zonder dat daar TypeScript-code voor nodig is.

2. `welcome.page.ts` (Het Brein)

- **Functie:** Dit beheert de logica van de welkomstpagina.
- **Wat het doet:** Dit is het TypeScript-bestand van het scherm. Omdat de knop in de HTML-blauwdruk de navigatie al helemaal zelf regelt, is dit bestand nu heel leeg en schoon. Het dient als basis om eventueel later slimme functies aan toe te voegen, zoals het automatisch selecteren van een taal of het starten van een opstart-animatie.

3. `welcome.page.scss` (De Stylist)

- **Functie:** Dit verzorgt de unieke vormgeving van het welkomstscherm.
- **Wat het doet:** Dit bestand regelt de exacte marges en uitlijning. Het zorgt ervoor dat de bloemenafbeelding netjes gecentreerd wordt, dat de titels de juiste lettergrootte krijgen en dat er voldoende witruimte rondom de inlogknop staat voor een rustige en professionele uitstraling.

4. `welcome-routing.module.ts` (De Verkeersregelaar)

- **Functie:** Dit regelt het toegangspad (de URL) naar de welkomstpagina.
- **Wat het doet:** Het vertelt Angular dat zodra de applicatie deze module opstart, direct de `WelcomePage` component geopend moet worden (via `path: ''`). Het zorgt ervoor dat deze pagina gekoppeld is aan het allereerste basispad van de app.

5. `welcome.module.ts` (De Gereedschapskist)

- **Functie:** Dit verzamelt alle benodigde bibliotheken voor de pagina.
- **Wat het doet:** Omdat Angular modulair werkt, vertelt dit bestand welke tools het welkomstscherm mag gebruiken. Het importeert de `IonicModule` (zodat Ionic-knoppen en elementen herkend worden) en de `WelcomePageRoutingModule` (de verkeersregelaar). Ook registreert het de pagina officieel binnen jouw app.

De map `welcome` vormt dus een onafhankelijk pakketje dat puur bedoeld is om niet-ingelogde gebruikers warm te ontvangen en ze soepel naar de inlogpagina te leiden.

welcome-routing.module.ts

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { WelcomePage } from './welcome.page';

const routes: Routes = [
  {
    path: '',
    component: WelcomePage
  }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule]
})
export class WelcomePageRoutingModule { }
```

welcome.module.ts

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { IonicModule } from '@ionic/angular';

import { WelcomePageRoutingModule } from './welcome-routing.module';
import { WelcomePage } from './welcome.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    WelcomePageRoutingModule
  ],
  declarations: [WelcomePage]
})
export class WelcomePageModule { }
```

welcome.page.html

```
<ion-content [fullscreen]="true" class="ion-text-center login-content-green">

  <div class="welcome-container ion-padding">
    <h1>Welkom op onze bedrijfspagina</h1>
    <p>Klik op de onderstaande knop om in te loggen op het Intranet.</p>

    <ion-button expand="block" routerLink="/login" color="primary"
class="login-button">
      Naar Inloggen
    </ion-button>
  </div>

</ion-content>
```

welcome.page.scss

```
// Styling voor de gecentreerde bloemenafbeelding bovenaan
.header-banner {
  display: block;
  margin: 0 auto;
  max-width: 90%;
  max-height: 250px;
  object-fit: cover;
  border-radius: 0 0 10px 10px;
}

// Styling voor de tekstcontainer onder de afbeelding
.welcome-container {
  margin-top: 20px;

  h1 {
    font-size: 1.8rem;
    font-weight: bold;
    // Voeg hier eventueel een kleur toe, bijv: color: #333;
  }

  p {
    font-size: 1rem;
    margin-top: 10px;
  }
}

// Styling voor de inlogknop
.login-button {
  margin-top: 30px;
}
```

welcome.page.ts

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-welcome',
  templateUrl: './welcome.page.html',
  styleUrls: ['./welcome.page.scss'],
  standalone: false,
})
export class WelcomePage implements OnInit {
  constructor() { }
  ngOnInit() { }
}
```

6. SIGNUP

De functie van dit bestand is het dienen als een **lichte doorgeef-service** (of *proxy*) specifiek voor de registratielogica. In plaats van dat jouw registratiescherm (`SignupPage`) rechtstreeks praat met de grote `LoginService`, loopt de communicatie via dit tussenstation.

Dit is de exacte functie van elk bestand in de map `signup`:

1. `signup.page.html` (De Blauwdruk)

- **Functie:** Dit bepaalt de structuur en de invoervelden op het registratiescherm.
- **Wat het doet:** Hierin staan de invoervelden voor het e-mailadres en het wachtwoord, de titel "Account aanmaken", de native terugknop (`<ion-back-button>`), de laad-spinner en de zachtrode foutbalk. Het zorgt via validaties (`required`, `minlength="6"`) dat gegevens al gecontroleerd worden vóórdat ze verzonden worden.

2. `signup.page.ts` (Het Brein)

- **Functie:** Dit regelt de logica en de afhandeling van de registratie.
- **Wat het doet:** Dit is het TypeScript-bestand dat luistert naar de knop "Registreren". Het start de asynchrone functie `onRegister()`, zet de laad-spinner aan, leest de invoer uit en stuurt deze via `await` naar de Signup service. Als het lukt, stuurt het de gebruiker naar de homepage; als het mislukt, vertaalt het de Firebase-foutcode naar een duidelijke Nederlandse melding.

3. `signup.page.scss` (De Stylist)

- **Functie:** Dit verzorgt de lokale vormgeving van het registratiescherm.
- **Wat het doet:** Dit bestand zorgt ervoor dat het formulier op grote schermen (zoals tablets) compact en gecentreerd blijft (`max-width: 500px`) en niet lelijk uitrekt. Het voegt extra witruimte (`margins`) toe tussen de invoervelden, stijlt de laad-spinner en verfijnt het uiterlijk van de rode foutbalk.

4. `signup-routing.module.ts` (De Verkeersregelaar)

- **Functie:** Dit regelt de URL-route naar de registratiepagina toe.
- **Wat het doet:** Het vertelt de applicatie dat zodra de app besluit de registratiemodule te openen, direct de `SignupPage` component opgestart moet worden (via `path: ""`). Het registreert deze route als een zogeheten *child*-route binnen de grote route-boom van je app.

5. `signup.module.ts` (De Gereedschapskist)

- **Functie:** Dit verzamelt alle benodigde bibliotheken voor deze pagina.
- **Wat het doet:** Omdat Angular modulair werkt, vertelt dit bestand welke tools het registratiescherm mag gebruiken. Het importeert de `IonicModule` (voor alle Ionic-knoppen), de `FormsModule` (onmisbaar om de getypte e-mail en het wachtwoord uit te lezen) en koppelt de boel aan de verkeersregelaar (routing). Ook registreert het de pagina officieel binnen je app.

6. `signup.ts` (De Doorgeef-Service)

- **Functie:** Dit dient als een tussenstation (proxy) voor de registratielogica.
- **Wat het doet:** Dit is een onzichtbare service op de achtergrond. Het vangt de e-mail en het wachtwoord op van de `SignupPage` en stuurt deze via een `return` door naar de grote `LoginService` (die de daadwerkelijke Firebase-cloud aanroept). Dit houdt de code flexibel voor toekomstige uitbreidingen.

7. signup.spec.ts (De Kwaliteitscontroleur)

- **Functie:** Dit is het geautomatiseerde testbestand van de Signup service.
- **Wat het doet:** Dit bestand bevat de testcode die je via de terminal uitvoert (met ng test). Het controleert buiten de browser om of de Signup service correct door Angular kan worden geïnitieerd en of de koppeling met de LoginService via een 'mock' (nep-object) vlekkeloos werkt.

De map signup bevat dus een compleet, zelfstandig pakket dat de gehele registratie-ervaring voor je gebruikers van A tot Z verzorgt.

signup-routing.module.ts

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { SignupPage } from './signup.page';

const routes: Routes = [
  {
    path: '',
    component: SignupPage
  }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})

export class SignupPageRoutingModule {}
```

signup.module.ts

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { IonicModule } from '@ionic/angular';

import { SignupPageRoutingModule } from './signup-routing.module';
import { SignupPage } from './signup.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
```

```

    SignupPageRoutingModule
  ],
  declarations: [SignupPage]
})
export class SignupPageModule {}

```

signup.page.html

```

<ion-header>
  <ion-toolbar color="primary">
    <ion-buttons slot="start">
      <!-- Knop om terug te gaan naar het inlogscherf -->
      <ion-back-button defaultHref="/login"></ion-back-button>
    </ion-buttons>
    <ion-title>Account aanmaken</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <!-- We koppelen de onRegister() functie aan de HTML submit -->
  <form #signupForm="ngForm" (ngSubmit)="onRegister(signupForm)">

    <ion-list lines="full" class="ion-margin-bottom">
      <!-- E-mail invoerveld -->
      <ion-item>
        <ion-input
          label="E-mailadres"
          labelPlacement="floating"
          type="email"
          ngModel
          name="email"
          required
          email
          #emailInput="ngModel"
        ></ion-input>
      </ion-item>

      <!-- Wachtwoord invoerveld -->
      <ion-item>
        <ion-input
          label="Wachtwoord (min. 6 tekens)"
          labelPlacement="floating"
          type="password"
          ngModel
          name="password"
          required
          minlength="6"
          #passwordInput="ngModel"
        ></ion-input>
      </ion-item>

```

```

    </ion-list>

    <!-- Foutmeldingen live tonen als Firebase een error geeft -->
    <div *ngIf="errorMessage" class="ion-padding ion-margin-bottom"
style="color: var(--ion-color-danger); background: rgba(var(--ion-color-
danger-rgb), 0.1); border-radius: 8px;">
        <p class="ion-no-margin">{{ errorMessage }}</p>
    </div>

    <!-- Registratieknop met ingebouwde laad-spinner -->
    <ion-button expand="block" type="submit" [disabled]="!signupForm.valid ||
isSigningUp">
        <ion-spinner *ngIf="isSigningUp" name="crescent"></ion-spinner>
        <span *ngIf="!isSigningUp">Registreren</span>
    </ion-button>

</form>
</ion-content>

```

signup.page.scss

```

// Centreer het formulier verticaal op grotere schermen
form {
    max-width: 500px;
    margin: 20px auto;
    display: flex;
    flex-direction: column;
}

// Geef de invoervelden wat extra ademruimte
ion-item {
    --padding-start: 8px;
    --padding-end: 8px;
    margin-bottom: 12px;
    border-radius: 8px;
}

// Styling voor de foutmeldingscontainer
div[style*="color: var(--ion-color-danger)"] {
    font-size: 0.9rem;
    font-weight: 500;
    border: 1px solid rgba(var(--ion-color-danger-rgb), 0.2);
}

// Geef de laad-spinner binnen de knop de juiste tussenruimte
ion-spinner {
    margin-right: 8px;
}

```

signup.page.ts

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { NgForm } from '@angular/forms';
import { Signup } from '../signup';

@Component({
  selector: 'app-signup',
  templateUrl: './signup.page.html',
  styleUrls: ['./signup.page.scss'],
  standalone: false
})
export class SignupPage {
  isSigningUp: boolean = false;
  errorMessage: string | null = null;

  constructor(
    private signupService: Signup,
    private router: Router
  ) { }

  // 1. Functie asynchroon gemaakt met 'async'
  async onRegister(form: NgForm) {
    if (!form.valid) {
      this.errorMessage = 'Vul alle velden correct in.';
      return;
    }

    this.errorMessage = null;
    this.isSigningUp = true;

    const email = form.value.email;
    const password = form.value.password;

    try {
      // 2. Wacht tot de Firebase registratie is voltooid
      const userCredential = await this.signupService.createNewUser(email,
password);
      console.log('Gebruiker succesvol geregistreerd:', userCredential.user);

      // 3. Navigeer naar de homepage en reset het formulier
      await this.router.navigateByUrl('/homepage');
      form.reset();
    } catch (error: any) {
      // 4. Vang de Firebase fouten op
      console.error('Firebase Registratie Fout:', error);
      this.errorMessage = this.getFriendlyErrorMessage(error.code);
    }
  }
}
```

```

    } finally {
      // 5. Schakel de spinner altijd uit aan het einde
      this.isSigningUp = false;
    }
  }

  private getFriendlyErrorMessage(errorCode: string): string {
    switch (errorCode) {
      case 'auth/email-already-in-use':
        return 'Dit e-mailadres is al in gebruik door een andere gebruiker.';
      case 'auth/invalid-email':
        return 'Het ingevulde e-mailadres is niet geldig.';
      case 'auth/weak-password':
        return 'Het wachtwoord is te zwak. Kies een wachtwoord van minimaal 6 tekens.';
      default:
        return 'Er is een fout opgetreden bij de registratie. Probeer het opnieuw.';
    }
  }
}

```

signup.spec.ts

```

import { TestBed } from '@angular/core/testing';
import { Signup } from './signup';
import { LoginService } from './login.service';

describe('Signup', () => {
  let service: Signup;

  beforeEach(() => {
    TestBed.configureTestingModule({
      // We vertellen de testomgeving dat LoginService bestaat
      providers: [
        Signup,
        { provide: LoginService, useValue: {} }
      ]
    });
    service = TestBed.inject(Signup);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });
});

```

signup.ts

```
import { Injectable, inject } from '@angular/core';
import { LoginService } from '../login.service';

@Injectable({
  providedIn: 'root',
})
export class Signup {
  private loginService = inject(LoginService);

  constructor() {}

  createNewUser(email: string, password: string) {
    return this.loginService.register(email, password);
  }
}
```

7. LOGIN

Dit is de functie van elk bestand in de map login:

1. login.guard.ts (De Poortwachter)

- **Functie:** Dit bestand regelt de **beveiliging** van je applicatie.
- **Wat het doet:** Dit is een onzichtbaar script (CanActivateFn) dat controleert of een gebruiker wel het recht heeft om een bepaalde pagina (zoals de /homepage) te bezoeken. Het praat direct met de LoginService om te kijken of Firebase een actieve, ingelogde gebruiker herkent.
 - *Indien ingelogd:* De guard opent geruisloos de deur (return true).
 - *Indien niet ingelogd:* De guard blokkeert de toegang, geeft een waarschuwing in de console en kaapt de route om de gebruiker direct terug te sturen naar het inlogschermb (/login).

2. login.guard.spec.ts (De Kwaliteitscontroleur)

- **Functie:** Dit is het **geautomatiseerde testbestand** voor de poortwachter.
- **Wat het doet:** Net als de andere .spec.ts bestanden is dit bedoeld voor softwaretests via de terminal (met ng test). Het bootst een mini-Angular omgeving na in het geheugen en controleert of de guard correct opstart en niet crasht door bijvoorbeeld een missende import. Dit zorgt ervoor dat de beveiliging van je intranet gegarandeerd blijft werken als je later andere delen van de app aanpast.

Samen zorgen deze twee bestanden er dus voor dat je intranet waterdicht is afgeschermd.

3. login.page.html (De Blauwdruk)

- **Functie:** Dit bepaalt de structuur en de elementen op het inlogschermb.
- **Wat het doet:** Hierin staan de invoervelden voor e-mail en wachtwoord, de bloemenafbeelding, de login-knop, de laad-spinner en het kader voor de foutmelding. Het zorgt ervoor dat Angular de getypte tekst kan verzamelen zodra de gebruiker op "Log in" drukt.

4. login.page.ts (Het Brein)

- **Functie:** Dit regelt de logica en de communicatie met Firebase.
- **Wat het doet:** Dit is het TypeScript-bestand dat luistert naar het formulier. Het controleert of de velden geldig zijn ingevuld en stuurt de inlogpoging asynchroon (async/await) door naar Firebase via de LoginService. Als het inloggen lukt, stuurt het de gebruiker naar de homepage; als het mislukt, vertaalt het de foutcode naar een nette Nederlandse melding.

5. login.page.scss (De Stylist)

- **Functie:** Dit verzorgt de unieke visuele styling van het inlogschermb.
- **Wat het doet:** Dit bestand overschrijft de standaard witte Ionic-stijlen. Het maakt de achtergrond een mooie groen-witte verloopkleur (linear-gradient), rondt de hoeken van de invoervelden en de banner mooi af, maakt de letters van de labels extra dik en stijlt de laad-spinner en de rode foutbalk.

6. login.routing.module.ts (De Verkeersregelaar)

- **Functie:** Dit regelt de URL-route naar de inlogpagina toe.
- **Wat het doet:** Het vertelt de applicatie dat zodra de app besluit de inlogmodule te openen, direct de LoginPage component opgestart moet worden (via path: "). Het registreert deze route als een zogeheten *child*-route binnen het grotere netwerk van je app.

7. login.module.ts (De Gereedschapskist)

- **Functie:** Dit verzamelt alle benodigde bibliotheken voor de pagina.
- **Wat het doet:** Omdat Angular modulair werkt, vertelt dit bestand welke tools het inlogscherf mag gebruiken. Het importeert de IonicModule (voor alle Ionic-knoppen), de FormsModule (onmisbaar om de getypte e-mail en het wachtwoord uit te lezen) en koppelt de boel aan de verkeersregelaar (routing). Ook registreert het de pagina officieel binnen je app.

8. login.page.spec.ts (De Kwaliteitscontroleur)

- **Functie:** Dit is het geautomatiseerde testbestand van de inlogpagina.
- **Wat het doet:** Dit bestand bevat code om op de achtergrond (buiten de browser om) te testen of de LoginPage correct door Angular gebouwd kan worden zonder vast te lopen. Het zorgt ervoor dat je app-beveiliging stabiel blijft als je later aanpassingen doet aan je code.

LOGIN-scherf

- **Eigen Layout:** Het inlogscherf heeft zijn eigen unieke visuele stijl (de grote welkomstkaart en de groen-witte verloopachtergrond) om gebruikers te verwelkomen. [1]
- **Geen Navigatie:** Op het inlogscherf mogen de tabbladen onderin en de Home-knop bovenin nog helemaal niet zichtbaar zijn, omdat de gebruiker nog niet is geïdentificeerd. [1]
- **Eigen Banner:** De bloemenbanner die in login.page.html staat, is specifiek ontworpen om binnen dat inlogformulier te passen.

login.guard.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { CanActivateFn } from '@angular/router';
import { loginGuard } from './login.guard';

describe('loginGuard', () => {
  const executeGuard: CanActivateFn = (...guardParameters) =>
    TestBed.runInInjectionContext(() => loginGuard(...guardParameters));

  beforeEach(() => {
    TestBed.configureTestingModule({});
  });

  it('should be created', () => {
    expect(executeGuard).toBeTruthy();
  });
});
```


login.guard.ts

```
import { inject } from '@angular/core';
import { Router, CanActivateFn } from '@angular/router';
import { LoginService } from '../login.service';
import { map, take } from 'rxjs/operators'; // Toegevoegd voor RxJS operaties

export const loginGuard: CanActivateFn = (route, state) => {
  const loginService = inject(LoginService);
  const router = inject(Router); // Toegevoegd om de gebruiker door te sturen

  // We luisteren live naar de Firebase datastroom van de LoginService
  return loginService.userIsAuthenticated$.pipe(
    take(1), // Pak één keer de huidige status en sluit de meting (veilig voor guards)
    map(isAuthenticated => {
      if (isAuthenticated) {
        return true; // Toegang toegestaan, de router navigeert nu door!
      }

      // Als je niet bent ingelogd, sturen we de gebruiker naar de inlogpagina
      console.warn('Navigatie geblokkeerd door guard: Niet ingelogd.');
      router.navigateByUrl('/login'); // Voorkomt dat de app op een wit scherm blijft hangen
      return false;
    })
  );
};
```

login.module.ts

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
//registreerfunctionaliteit
import { IonicModule } from '@ionic/angular';
import { LoginPageRoutingModule } from './login.routing.module';

import { LoginPage } from './login.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    LoginPageRoutingModule
  ],
  declarations: [LoginPage]
})
export class LoginPageModule {}
```

login.page.html

```
<ion-content [fullscreen]="true" class="ion-padding login-content-green"
[scrolly]="true">

  <form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)" class="full-
width-form login-container-flex">
    <ion-grid class="ion-no-padding full-height-grid">

<!-- Bloemenafbeelding -->
      <ion-row class="image-row">
        <ion-col size="12" class="ion-no-padding">
          
        </ion-col>
      </ion-row>

<!-- Welkomstkaart -->
      <ion-row class="form-row">
        <ion-col size="10" offset="1">
          <ion-card class="ion-no-margin ion-padding login-welcome-card-green
full-width-card">
            <h2>Intranet</h2>
            <p class="welcome-text">Vul je gegevens in om toegang te krijgen
tot het Intranet</p>
          </ion-card>
        </ion-col>
      </ion-row>

<!-- E-mail Veld -->
      <ion-row class="form-row">
        <ion-col size="10" offset="1">
          <ion-item class="input-item-green">
            <ion-input label-placement="floating" type="email" ngModel
name="email" required email>
              <div slot="label" style="font-size: 16px; font-weight: 600;
color: rgba(0, 0, 0, 0.5);">E-mail</div>
            </ion-input>
          </ion-item>
        </ion-col>
      </ion-row>

<!-- Wachtwoord Veld met het oogje -->
      <ion-row class="form-row">
        <ion-col size="10" offset="1">
          <ion-item class="input-item-green">
            <ion-input label-placement="floating" type="password" ngModel
name="password" required>
              <!-- Gecorrigeerde kleurwaarde (rgba) -->
            </ion-input>
          </ion-item>
        </ion-col>
      </ion-row>
    </ion-grid>
  </form>
</ion-content>
```

```

        <div slot="label" style="font-size: 20px; font-weight: 600;
color: rgba(0, 0, 0, 0.5);">Wachtwoord</div>
        <ion-input-password-toggle slot="end"></ion-input-password-
toggle>
    </ion-input>
</ion-item>
</ion-col>
</ion-row>

<!-- Foutmelding -->
    <ion-row *ngIf="errorMessage" class="error-row">
        <ion-col size="10" offset="1" class="ion-text-center login-error-
status">
            <ion-icon name="alert-circle-outline"></ion-icon>
            <p>{{ errorMessage }}</p>
        </ion-col>
    </ion-row>

<!-- Login Knop -->
    <ion-row class="button-row">
        <ion-col size="10" offset="1">
            <ion-button expand="block" class="login-btn-submit-green"
type="submit" [disabled]="!loginForm.valid || isLoggingIn">
                Log in
            </ion-button>
        </ion-col>
    </ion-row>

<!-- Inlog Melding met spinner -->
    <ion-row *ngIf="isLoggingIn" class="loading-row">
        <ion-col size="10" offset="1" class="ion-text-center login-loading-
status">
            <ion-spinner name="crescent"></ion-spinner>
            <p>You are being logged in...</p>
        </ion-col>
    </ion-row>

<!-- Sign up Knop -->
    <ion-row class="button-row">
        <ion-col size="10" offset="1">
            <!-- Click-event toegevoegd -->
            <ion-button expand="block" class="login-btn-submit-green"
type="button" (click)="onSignUp()">
                Sign up
            </ion-button>
        </ion-col>
    </ion-row>

<!-- Wachtwoord Vergeten -->

```

```

        <ion-row class="forgot-password-row">
          <ion-col size="10" offset="1" class="ion-text-center">
            <a (click)="onForgotPassword()" class="forgot-password-
link">Wachtwoord vergeten?</a>
          </ion-col>
        </ion-row>

      </ion-grid>
    </form>
  </ion-content>

```

login.page.scss

```

/* Globale container achtergrond */
.login-content-green {
  --padding-top: 13px;
  --padding-bottom: 13px;
  --padding-start: 13px;
  --padding-end: 13px;
  --background: linear-gradient(to bottom, #a2d2a4, #e8f5e9) !important;
}

/* Formulier hoogte zonder scrollen */
.login-container-flex {
  height: 100%;
}

.full-height-grid {
  height: 100%;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
}

/* Afbeelding */
.image-row {
  width: 100%;
  margin: 0;
}

.login-banner-wide {
  width: 100% !important;
  max-height: 18vh;
  height: 200px;
  display: block;
  object-fit: cover;
  border-bottom-left-radius: 24px;
  border-bottom-right-radius: 24px;
}

```

```

/* Welkomstkaart */
.login-welcome-card-green {
  margin-bottom: 16px;

  h2 {
    margin-top: 0;
    font-family: var(--ion-font-family), system-ui, -apple-system, sans-serif !important;
    font-weight: 700;
    letter-spacing: -0.02em;
    color: black;
  }
}

.full-width-card {
  margin-left: 0 !important;
  margin-right: 0 !important;
  width: 100% !important;
  box-shadow: none;
}

.welcome-text {
  margin-bottom: 0;
  font-family: Georgia, 'Times New Roman', Times, serif;
  color: black;
}

/* Input velden basis container */
.input-item-green {
  --background: rgba(255, 255, 255, 0.6) !important;
  --border-radius: 8px;
  --padding-start: 16px;
  --highlight-color-focused: var(--ion-color-primary, #2e6930) !important;
  margin-bottom: 15px;
}

/* Tekst invoerveld normaal */
.input-item-green,
.input-item-green ion-input,
.input-item-green ::part(label),
.input-item-green ::part(native) {
  --color: #000000 !important;
  color: #000000 !important;
  --font-size: 1rem !important;
  font-size: 1rem !important;
  font-family: var(--ion-font-family), system-ui, -apple-system, sans-serif !important;
  --font-weight: 400 !important;
  font-weight: 400 !important;
}

```

```

    letter-spacing: normal !important;
}

/* Tekst placeholder */
.input-item-green ion-input::part(placeholder) {
    --placeholder-color: rgba(0, 0, 0, 0.5) !important;
    font-size: large;
    font-weight: bold;
    color: rgba(0, 0, 0, 0.5) !important;
}

/* Styling voor de labels 'E-mail' en 'Wachtwoord' */
.input-item-green ion-input {
    /* Kleur voor het label */
    --label-color: rgba(0, 0, 0, 0.5) !important;
}

/* Bereik de binnenkant van de Shadow DOM voor tekst-grootte en dikte */
.input-item-green ion-input::part(label-text-wrapper),
.input-item-green ion-input::part(label-text) {
    font-size: large !important;
    font-weight: 800 !important;
}

/* Wachtwoord oogje */
ion-input-password-toggle {
    --color: #000000 !important;
    --color-focused: #000000 !important;
    color: #000000 !important;
}

/* Tekst knoppen bold */
.login-btn-submit-green,
.login-btn-signup-green {
    --color: #000000 !important;
    color: #000000 !important;
    --font-size: 1rem !important;
    font-size: 1rem !important;
    font-family: var(--ion-font-family), system-ui, -apple-system, sans-serif !important;
    --font-weight: 700 !important;
    font-weight: 700 !important;
    letter-spacing: -0.02em !important;
    text-transform: none !important;
    --background: rgba(255, 255, 255, 0.8) !important;
    --border-radius: 8px !important;
    margin-top: 5px;
}

```

```

/* Marges voor rijen */
.form-row {
  margin-bottom: 8px;
}

.button-row {
  margin-top: 12px;
}

.loading-row,
.error-row {
  margin-top: 5px;
  margin-bottom: 5px;
}

/* Wachtwoord vergeten */
.forgot-password-row {
  margin-top: 8px;
  margin-bottom: 16px;

  .forgot-password-link {
    color: var(--ion-color-medium);
    text-decoration: none;
    font-size: 0.9rem;
    cursor: pointer;

    &:hover {
      text-decoration: underline;
    }
  }
}

/* Laden en spinner */
.login-loading-status {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
  margin-top: 10px;

  ion-spinner {
    --color: #000000 !important;
    width: 24px;
    height: 24px;
  }

  p {
    margin: 0;
  }
}

```

```

    color: #000000 !important;
    font-size: 1rem !important;
    font-family: var(--ion-font-family), system-ui, -apple-system, sans-serif !important;
    font-weight: 700 !important;
    letter-spacing: -0.02em !important;
  }
}

/* Foutmelding */
.login-error-status {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  background-color: rgba(255, 235, 235, 0.9);
  padding: 10px;
  border-radius: 8px;
  border: 1px solid var(--ion-color-danger, #eb445a);

  ion-icon {
    color: var(--ion-color-danger, #eb445a);
    font-size: 1.3rem;
  }

  p {
    margin: 0;
    color: var(--ion-color-danger, #eb445a) !important;
    font-size: 0.9rem !important;
    font-family: var(--ion-font-family), system-ui, -apple-system, sans-serif !important;
    font-weight: 700 !important;
    letter-spacing: -0.01em !important;
    text-align: left;
  }
}

```

login.page.spec.ts

```

import { ComponentFixture, TestBed } from '@angular/core/testing';
import { LoginPage } from '../login.page';
import { LoginService } from '../login.service'; // Importeer je service
import { Router } from '@angular/router'; // Importeer de router
import { FormsModule } from '@angular/forms'; // Nodig omdat je NgForm gebruikt
import { IonicModule } from '@ionic/angular'; // Nodig voor de ion-componenten

describe('LoginPage', () => {
  let component: LoginPage;
  let fixture: ComponentFixture<LoginPage>;

```



```

let mockLoginService: any;
let mockRouter: any;

beforeEach(async () => {
  // 1. Maak 'nep' (mock) objecten aan voor de nodige functionaliteiten
  mockLoginService = {
    login: jasmine.createSpy('login').and.returnValue(Promise.resolve())
  };
  mockRouter = {
    navigateByUrl:
jasmine.createSpy('navigateByUrl').and.returnValue(Promise.resolve(true))
  };

  // 2. Configureer de testmodule met alle benodigde tools en providers
  await TestBed.configureTestingModule({
    declarations: [ LoginPage ],
    imports: [
      FormsModule,
      IonicModule.forRoot() // Zorgt dat Ionic elementen compileren in de
test
    ],
    providers: [
      { provide: LoginService, useValue: mockLoginService }, // Lever de
nep-service
      { provide: Router, useValue: mockRouter } // Lever de nep-router
    ]
  }).compileComponents();

  fixture = TestBed.createComponent(LoginPage);
  component = fixture.componentInstance;
  fixture.detectChanges();
});

it('should create', () => {
  expect(component).toBeTruthy(); // Deze test kleurt nu weer prachtig
groen!
});
});

```

login.page.ts

```

import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { NgForm } from '@angular/forms';
import { LoginService } from '../login.service';
import { AlertController } from '@ionic/angular'; // AlertController
geïmporteerd

@Component({
  selector: 'app-login',

```

```

templateUrl: './login.page.html',
styleUrls: ['./login.page.scss'],
standalone: false,
}))
export class LoginPage {
  isLoggedIn: boolean = false;
  errorMessage: string | null = null;

  constructor(
    private loginService: LoginService,
    private router: Router,
    private alertController: AlertController // AlertController geïnjecteerd
  ) { }

  async onSubmit(form: NgForm) {
    if (!form.valid) {
      console.log('Formulier is ongeldig. Controleer de velden.');
```

this.errorMessage = 'Vul alle verplichte velden correct in.';

return;

}

this.errorMessage = null;

this.isLoggedIn = true;

const email = form.value.email;

const password = form.value.password;

console.log('Inlogpoging voor:', email);

try {

 await this.loginService.login(email, password);

 await this.router.navigateByUrl('/homepage');

 form.reset();

} catch (error: any) {

 console.error('Firebase login fout:', error);

 if (

 error.code === 'auth/wrong-password' ||

 error.code === 'auth/user-not-found' ||

 error.code === 'auth/invalid-credential'

) {

 this.errorMessage = 'Onjuist e-mailadres of wachtwoord.';

 } else if (error.code === 'auth/invalid-email') {

 this.errorMessage = 'Het ingevoerde e-mailadres is niet geldig.';

 } else {

 this.errorMessage = 'Inloggen mislukt. Probeer het later opnieuw.';

 }

} finally {

```

        this.isLoggingIn = false;
    }
}

onSignUp() {
    console.log('Navigeer naar registratiepagina');
    this.router.navigateByUrl('/signup');
}

// Geactiveerde wachtwoordherstel met een Ionic pop-up alert
async onForgotPassword() {
    console.log('Wachtwoord vergeten geactiveerd via alert dialoog');

    const alert = await this.alertController.create({
        header: 'Wachtwoord herstellen',
        message: 'Vul je e-mailadres in om een herstellink te ontvangen.',
        inputs: [
            {
                name: 'email',
                type: 'email',
                placeholder: 'jouw-email@domein.nl'
            }
        ],
        buttons: [
            {
                text: 'Annuleren',
                role: 'cancel'
            },
            {
                text: 'Versturen',
                handler: async (data) => {
                    if (!data.email || data.email.trim() === '') {
                        this.errorMessage = 'Vul een geldig e-mailadres in.';
                        return;
                    }

                    try {
                        this.errorMessage = null;
                        // Roep de nieuwe methode uit je LoginService aan
                        await this.loginService.sendPasswordReset(data.email);

                        // Toon succesmelding aan de gebruiker
                        const successAlert = await this.alertController.create({
                            header: 'E-mail verzonden',
                            message: 'Er is een link gestuurd naar ' + data.email + ' om
je wachtwoord te herstellen. Controleer ook je spambox.',
                            buttons: ['OK']
                        });
                        await successAlert.present();
                    }
                }
            }
        ]
    });
}

```

```

    } catch (error: any) {
        console.error('Reset e-mail fout:', error);

        if (error.code === 'auth/user-not-found' || error.code ===
'auth/invalid-credential') {
            this.errorMessage = 'Dit e-mailadres is niet bekend in ons
systeem.';
        } else if (error.code === 'auth/invalid-email') {
            this.errorMessage = 'Het ingevoerde e-mailadres is niet
geldig.';
        } else {
            this.errorMessage = 'Kon de herstellink niet verzenden.
Probeer het later opnieuw.';
        }
    }
}
}
]
});

    await alert.present();
}
}

```

login.routing.module.ts

```

import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { LoginPage } from './login.page';

const routes: Routes = [
    {
        path: '',
        component: LoginPage
    }
];

@NgModule({
    imports: [RouterModule.forChild(routes)],
    exports: [RouterModule],
})
export class LoginPageRoutingModule {}

```

7.1. Loginservice

1. login.service.ts (De Centrale Database-Koppeling)

- **Functie:** Dit bestand regelt de **directe communicatie met de Firebase-cloud** voor je hele applicatie.
- **Wat het doet:** Dit is een centrale TypeScript-klasse die door de hele app gedeeld wordt (providedIn: 'root'). In plaats van dat elke pagina zelf verbinding probeert te maken met internet, regelt dit bestand dat centraal. Het bevat de functies voor:
 - **Inloggen** (signInWithEmailAndPassword) [29, 31]
 - **Registreren** (createUserWithEmailAndPassword)
 - **Uitloggen** (signOut)
 - **Authenticatie-status:** Het houdt via een RxJS-datastroom (currentUser\$) live bij of er op dit moment een gebruiker is ingelogd, zodat de loginGuard direct weet of de deur naar de homepage open mag.

2. login.spec.ts (De Kwaliteitscontroleur)

- **Functie:** Dit is het **geautomatiseerde testbestand** voor de service.
- **Wat het doet:** Dit bestand bevat de testcode die je via de terminal uitvoert (met ng test). Dankzij de *mock* (het nep-Firebase object) controleert dit bestand buiten de browser om of de LoginService correct door Angular kan worden opgestart en geïnjecteerd zonder fouten. Dit garandeert dat de vitale basisfuncties (zoals inloggen en uitloggen) blijven werken als je later updates uitvoert aan je app.

login.service.ts

```
import { Injectable, inject } from '@angular/core';
import { Auth, createUserWithEmailAndPassword, signInWithEmailAndPassword,
signOut, user, sendPasswordResetEmail } from '@angular/fire/auth'; //
sendPasswordResetEmail toegevoegd
import { map, Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class LoginService {
  // 1. Injecteer de Firebase Auth module
  private auth: Auth = inject(Auth);

  // 2. Firebase houdt hier live de ingelogde gebruiker in bij
  currentUser$ = user(this.auth);

  constructor() { }

  // 3. Je Guard kan deze getter blijven gebruiken!
  // We kijken of er een geldige gebruiker (user) actief is in Firebase
```

```

get userIsAuthenticated$(): Observable<boolean> {
  return this.currentUser$.pipe(
    map(user => user !== null) // Geeft true als er een user is, anders
false
  );
}

// 4. Nieuw account registreren in Firebase
register(email: string, password: string) {
  return createUserWithEmailAndPassword(this.auth, email, password);
}

// 5. Inloggen via Firebase (vervangt jouw oude handmatige login)
login(email: string, password: string) {
  return signInWithEmailAndPassword(this.auth, email, password);
}

// 6. Uitloggen via Firebase (vervangt jouw oude handmatige logout)
logout() {
  return signOut(this.auth);
}

// 7. Wachtwoord herstel-mail versturen via Firebase
sendPasswordReset(email: string) {
  return sendPasswordResetEmail(this.auth, email);
}
}

```

login.spec.ts

```

import { TestBed } from '@angular/core/testing';
import { LoginService } from './login.service'; // 1. Correcte naam
geïmporteerd
import { Auth } from '@angular/fire/auth'; // 2. Auth geïmporteerd voor de
testomgeving

describe('LoginService', () => {
  let service: LoginService;
  let mockAuth: any;

  beforeEach(() => {
    // 3. We maken een 'nep' Firebase Auth object aan zodat de test niet echt
internet op hoeft
    mockAuth = {};

    TestBed.configureTestingModule({
      providers: [
        LoginService,

```

```

        { provide: Auth, useValue: mockAuth } // 4. Lever de nep-Auth aan de
service
    ]
    });

    service = TestBed.inject(LoginService); // 5. Injecteer de correcte
service
    });

    it('should be created', () => {
        expect(service).toBeTruthy(); // Deze test kleurt nu weer perfect groen!
    });
});

```

8. HOMEPAGE

Deze map **homepage**, regelt de hoofd-layout en de tabbladenstructuur van je intranet. Elk bestand in deze map heeft een specifieke, unieke taak. Ze werken nauw samen om ervoor te zorgen dat de bovenbalk, de bloemenbanner en de onderbalk met tabbladen correct op het scherm verschijnen en functioneren.

Dit is wat elk bestand in de map homepage precies doet:

1. homepage.page.html (De Blauwdruk)

- **Wat het doet:** Dit bestand bepaalt de vaste structuur van je dashboard.
- **Functie:** Het bouwt de vaste bovenbalk (<ion-header>) met de Home-knop en de Uitlogknop. Ook plaatst het de centrale bloemenafbeelding (.header-banner) zodat deze boven elke pagina zichtbaar blijft, en herbergt het de onderbalk (<ion-tabs>) waarmee je tussen de pagina's kunt wisselen.

2. homepage.page.ts (Het Brein)

- **Wat het doet:** Dit bestand regelt de intelligentie en de acties van de hoofd-layout.
- **Functie:** Hierin staat de TypeScript-logica. Het luistert naar de Uitlogknop uit de HTML-blauwdruk. Zodra een gebruiker daarop klikt, start de functie onLogout(). Deze functie beëindigt asynchroon de sessie in de Firebase-cloud en stuurt de gebruiker via de router veilig terug naar het beginscherm.

3. homepage.page.scss (De Stylist)

- **Wat het doet:** Dit bestand regelt de specifieke vormgeving van de hoofdcontainer.
- **Functie:** Het bevat de CSS/SCSS-opmaak die specifiek voor dit frame geldt. De belangrijkste taak van dit bestand is de geavanceerde *scroll-fix* (ion-content en ::part(scroll)). Dit zorgt ervoor dat de inhoud van je tabs soepel scrollt en dat de app op iPhones (Safari) niet ongewenst op en neer veert. Alsook dat de knoppenmatrix niet verticaal wordt platgedrukt.

4. homepage-routing.module.ts (De Verkeersregelaar)

- **Wat het doet:** Dit bestand regelt de paden (URL-structuur) van je dashboard.
- **Functie:** Het definieert de hoofdroute /homepage en beheert de kinderroutes (children). Het vertelt Angular dat de subpagina's (main, page1, page2, page3) via *lazy loading* binnen de <ion-tabs> geladen moeten worden. Ook zorgt het voor de automatische doorverwijzing naar /homepage/main als iemand de pagina opent.

5. homepage.module.ts (De Gereedschapskist)

- **Wat het doet:** Dit bestand verzamelt alle benodigde bibliotheken en onderdelen.
- **Functie:** Omdat Angular pagina's los van elkaar inlaadt, vertelt dit bestand aan Angular welk gereedschap de hoofdcontainer nodig heeft. Het importeert de IonicModule (zodat tags zoals <ion-tabs> en <ion-icon> herkend worden) en koppelt de module aan de verkeersregelaar (routing). Omdat je project werkt met standalone: false, registreert dit bestand de HomePagePage component officieel binnen de app.

De map homepage vormt dus de complete, stabiele "omgeving" waarin jouw app leeft zodra een gebruiker is ingelogd.

homepage-routing.module.ts

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomepagePage } from './homepage.page';

const routes: Routes = [
  {
    path: '',
    component: HomepagePage,
    children: [
      {
        path: 'main',
        loadChildren: () => import('../home/home.module').then(m =>
m.HomePageModule)
      },
      {
        path: 'page1',
        loadChildren: () => import('../page1/page1.module').then(m =>
m.Page1PageModule)
      },
      {
        path: 'page2',
        loadChildren: () => import('../page2/page2.module').then(m =>
m.Page2PageModule)
      },
      {
        path: 'page3',
        loadChildren: () => import('../page3/page3.module').then(m =>
m.Page3PageModule)
      },
      {
        path: '',
        redirectTo: 'main',
        pathMatch: 'full'
      }
    ]
  }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule]
})
export class HomepageRoutingModule { }
```

homepage.module.ts

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { IonicModule } from '@ionic/angular';

import { HomepageRoutingModule } from './homepage-routing.module';
import { HomepagePage } from './homepage.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    HomepageRoutingModule
  ],
  declarations: [HomepagePage]
})
export class HomePageModule { }
```

homepage.page.html

```
<ion-header [translucent]="true">
  <ion-toolbar color="primary">
    <!-- Dynamische titel links/midden -->
    <ion-title>{{ activePageTitle }}</ion-title>

    <!-- Uitlogknop aan de rechterkant van de balk -->
  <ion-buttons slot="end">
    <ion-button (click)="onLogout()" class="logout-header-button">
      <!-- Tekst staat nu netjes vóór het icoon -->
      <ion-label class="logout-text">Uitloggen</ion-label>
      <ion-icon slot="end" name="log-out-outline"></ion-icon>
    </ion-button>
  </ion-buttons>
</ion-toolbar>
</ion-header>

<ion-tabs>
  <!-- De router-outlet MOET binnen ion-tabs staan -->
  <ion-router-outlet></ion-router-outlet>

  <ion-tab-bar slot="bottom">
    <ion-tab-button tab="main" href="/homepage/main">
      <ion-icon name="home"></ion-icon>
      <ion-label>Home</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="page1" href="/homepage/page1">
      <ion-icon name="apps"></ion-icon>
```

```

        <ion-label>Pagina 1</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="page2" href="/homepage/page2">
        <ion-icon name="document"></ion-icon>
        <ion-label>Pagina 2</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="page3" href="/homepage/page3">
        <ion-icon name="person"></ion-icon>
        <ion-label>Pagina 3</ion-label>
    </ion-tab-button>

    <!-- Uitlogknop in de tab-balk -->
    <ion-tab-button (click)="onLogout()">
        <ion-icon name="log-out"></ion-icon>
        <ion-label>Uitloggen</ion-label>
    </ion-tab-button>
</ion-tab-bar>
</ion-tabs>

```

homepage.page.scss

```

/* Moderne, cross-browser scroll-fix voor alle browsers (Chrome, Firefox,
Safari, Edge) */
ion-content {
    --overflow: auto;
    overflow-behavior: contain; /* Voorkomt dat de hele app mee-veert bij het
scrollen */

    &::part(scroll) {
        overflow-y: auto !important;
        display: flex;
        flex-direction: column;
        height: 100%;
    }
}

/* Zorgt ervoor dat de matrix-grid de ruimte krijgt om verticaal uit te rekken
*/
.matrix-grid {
    flex-shrink: 0; /* Voorkomt dat de grid wordt samengedrukt door flexbox */
    height: auto !important;
    overflow: visible !important;
}

/* Extra marge boven de rode uitlogknop op de pagina zelf */
.logout-action-btn {
    margin-top: 16px;
}

```

```

/* Ruimte rondom de welkomsttekst */
.welcome-container {
  margin-bottom: 24px;
}

#container {
  text-align: center;

  position: absolute;
  left: 0;
  right: 0;
  top: 50%;
  transform: translateY(-50%);
}

#container strong {
  font-size: 20px;
  line-height: 26px;
}

#container p {
  font-size: 16px;
  line-height: 22px;

  color: #8c8c8c;

  margin: 0;
}

#container a {
  text-decoration: none;
}

```

homepage.page.spec.ts

```

import { ComponentFixture, TestBed } from '@angular/core/testing';
import { HomepagePage } from './homepage.page';

describe('HomepagePage', () => {
  let component: HomepagePage;
  let fixture: ComponentFixture<HomepagePage>;

  beforeEach(() => {
    fixture = TestBed.createComponent(HompagePage);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {

```

```

    expect(component).toBeTruthy();
  });
});

```

homepage.page.ts

```

import { Component, OnInit, OnDestroy } from '@angular/core';
import { Router, NavigationEnd } from '@angular/router';
import { Subscription } from 'rxjs';
import { filter } from 'rxjs/operators';
import { LoginService } from '../login.service'; // Pas het pad aan indien
nodig

@Component({
  selector: 'app-homepage',
  templateUrl: './homepage.page.html',
  styleUrls: ['./homepage.page.scss'],
  standalone: false,
})
export class HomepagePage implements OnInit, OnDestroy {
  activePageTitle: string = 'Home'; // Standaard titel bovenin
  private routerSub!: Subscription;

  constructor(
    private loginService: LoginService,
    private router: Router
  ) {}

  ngOnInit() {
    // 1. Luister naar route-veranderingen voor de dynamische titel
    this.routerSub = this.router.events.pipe(
      filter(event => event instanceof NavigationEnd)
    ).subscribe((event: any) => {
      this.updateTitle(event.urlAfterRedirects);
    });

    // 2. Voer de check direct uit bij het eerste laden van de pagina
    this.updateTitle(this.router.url);
  }

  // Methode om de juiste titel te bepalen op basis van de URL
  private updateTitle(url: string) {
    const urlSegments = url.split('/');
    const lastSegment = urlSegments[urlSegments.length - 1];

    switch (lastSegment) {
      case 'main':
        this.activePageTitle = 'Home';
        break;
    }
  }

```

```

    case 'page1':
        this.activePageTitle = 'Pagina 1';
        break;
    case 'page2':
        this.activePageTitle = 'Pagina 2';
        break;
    case 'page3':
        this.activePageTitle = 'Pagina 3';
        break;
    default:
        this.activePageTitle = 'Intranet';
    }
}

// Methode om de gebruiker uit te loggen via Firebase
async onLogout() {
    try {
        console.log('Gebruiker logt uit...');

        // Wacht tot Firebase de sessie heeft beëindigd
        await this.loginService.logout();

        // Stuur terug naar welcome en wis de historie (zodat ze niet 'terug'
        // kunnen klikken)
        await this.router.navigateByUrl('/welcome', { replaceUrl: true });

        console.log('Succesvol uitgelogd.');
```

```

    } catch (error) {
        console.error('Fout tijdens het uitloggen:', error);
    }
}

ngOnDestroy() {
    // Netjes afmelden om geheugenlekken te voorkomen
    if (this.routerSub) {
        this.routerSub.unsubscribe();
    }
}
}

```

9. PAGES

9.1. HOME

Dit is de specifieke functie van elk bestand binnen jouw **home**-map:

1. **home.page.html (De Blauwdruk)**

- **Functie:** Dit is de structuur van je pagina.
- **Wat het doet:** Hierin bepaal je welke elementen er op het scherm komen te staan, zoals teksten (<h2>), knoppen (<ion-button>) of rasters (<ion-grid>). Het bevat puur de content die de gebruiker te zien krijgt.

2. **home.page.ts (Het Brein)**

- **Functie:** Dit is de TypeScript-logica achter het scherm.
- **Wat het doet:** Dit bestand regelt de intelligentie van de pagina. Hoewel hij nu leeg is, is dit de plek waar je variabelen aanmaakt, data ophaalt uit Firebase, of functies schrijft die moeten starten zodra een gebruiker op een knop klikt.

3. **home.page.scss (De Stylist)**

- **Functie:** Dit is de vormgeving van deze specifieke pagina.
- **Wat het doet:** Hierin zet je de CSS-opmaak die *alleen* voor de home-pagina geldt (zoals de kleur van de teksten of specifieke marges). Omdat je de belangrijkste stijlen al in `global.scss` hebt staan, blijft dit bestand lekker klein.

4. **home-routing.module.ts (De Verkeersregelaar)**

- **Functie:** Dit regelt het pad (de URL) naar deze pagina.
- **Wat het doet:** Dit bestand vertelt Angular: *"Als de app vraagt naar de route main, laad dan de component HomePage in."* Het zorgt ervoor dat de pagina correct gekoppeld kan worden aan de onderbalk (tabs).

5. **home.module.ts (De Gereedschapskist)**

- **Functie:** Dit verzamelt alle onderdelen en bibliotheken die de pagina nodig heeft.
 - **Wat het doet:** Angular laadt pagina's apart in (*lazy loading*). Dit bestand vertelt Angular dat de home-pagina gebruik mag maken van de `IonicModule` (voor alle Ionic-knoppen), de `FormsModule` (voor formulieren) en dat de `HomePage` component hier officieel geregistreerd staat.
-

home-routing.module.ts

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomePage } from './home.page';

const routes: Routes = [
  {
    path: '',
    component: HomePage
  }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule]
})
export class HomePageRoutingModule { }
```

home.module.ts

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { IonicModule } from '@ionic/angular'; // <-- 1. Controleer deze import

import { HomePageRoutingModule } from './home-routing.module';
import { HomePage } from './home.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    HomePageRoutingModule
  ],
  declarations: [HomePage]
})
export class HomePageModule { }
```

home.page.html

```
<!-- Content en Scrollfunctie (Eén hoofdcontainer met jouw klassen) -->
<ion-content [fullscreen]="true" class="ion-padding native-scroll"
[scrollY]="true">

  <!-- Gekoppeld aan de .content wrapper voor breedtebeperking en centrering -
  ->
  <div class="content">

    <!-- Afbeelding met titel strak linksboven onder de foto -->
```



```

<div class="page-hero">
  
  <h2>Welkom op ons Intranet</h2>
</div>

<!-- De overige teksten en grids binnen de content wrapper -->
<p>Content</p>

<div class="matrix-grid">
  <!-- Hier komt je matrix-grid of overige content -->
</div>

</div>
</ion-content>

```

home.page.scss

```

/* =====
Algemene Tekststijlen (Binnen de content wrapper)
===== */
.content {
  // Verwijder max-width of zet deze heel hoog voor grote schermen (bijv. 100%
  // of 1400px)
  max-width: 100%;
  width: 95%; // Zorgt voor een kleine, gelijke marge (2.5%) aan de linker- en
  // rechterkant
  margin: 0 auto;

  // Haalt de padding uit de HTML en regelt het via CSS
  padding-top: 16px;
  padding-bottom: 16px;

  // Standaard paragraaf styling
  p {
    color: #555555;
    margin-bottom: 24px;
    font-size: 0.95rem;
    opacity: 0.9;
  }
}

/* =====
Afbeelding en Titel Layout (Nu ook pagina-breed)
===== */
.page-hero {
  display: flex;
  flex-direction: column;
  align-items: flex-start;

```

```

width: 100%;
margin-bottom: 16px;

.hero-image {
  width: 100%;
  max-height: 250px; // Iets verhoogd omdat de foto nu breder mag
  uitrekken
  object-fit: cover;
  border-radius: 8px;
  margin-bottom: 12px;
}

h2 {
  margin: 0;
  text-align: left !important;
  font-size: 24px;
  font-weight: 700;
  color: var(--ion-color-primary-shade, #38803b);
}

/*
=====
Kaarten en Overige Componenten
=====
= */
ion-card {
  margin-inline: 0px;

  img {
    width: 100%;
    display: block;
    border-radius: 12px 12px 0 0;
  }

  h2 {
    color: var(--ion-color-primary-shade, #38803b);
    margin-top: 16px;
    margin-bottom: 8px;
  }
}
}

```

home.page.ts

```

import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-home',
  templateUrl: './home.page.html',

```

```
    styleUrls: ['./home.page.scss'],
    standalone: false,
  })
export class HomePage implements OnInit {

  // De router en onLogout zijn hier niet meer nodig
  constructor() { }

  ngOnInit() { }
}
```

9.2. PAGE1

page1-routing.module.ts

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { Page1Page } from './page1.page';

const routes: Routes = [
  {
    path: '',
    component: Page1Page
  }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class Page1PageRoutingModule {}
```

page1.module.ts

```
import { NgModule, CUSTOM_ELEMENTS_SCHEMA } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { IonicModule } from '@ionic/angular';
import { Page1PageRoutingModule } from './page1-routing.module';
import { Page1Page } from './page1.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    Page1PageRoutingModule
  ],
  declarations: [Page1Page],
  schemas: [CUSTOM_ELEMENTS_SCHEMA]
})
export class Page1PageModule {}
```

page1.page.html

```
<ion-content [fullscreen]="true" class="ion-padding native-scroll"
[scrollY]="true">

  <div class="content">

    <div class="page-hero">
      
      <h2>Welkom op Page 1</h2>
    </div>

    <p>Content </p>

    <div class="matrix-grid">
    </div>

  </div>
</ion-content>
```

page1.page.scss

```
.content {
  max-width: 100%;
  width: 95%;
  margin: 0 auto;

  padding-top: 16px;
  padding-bottom: 16px;

  p {
    color: #555555;
    margin-bottom: 24px;
    font-size: 0.95rem;
    opacity: 0.9;
  }

  .page-hero {
    display: flex;
    flex-direction: column;
    align-items: flex-start;
    width: 100%;
    margin-bottom: 16px;

    .hero-image {
      width: 100%;
      max-height: 250px;
      object-fit: cover;
      border-radius: 8px;
      margin-bottom: 12px;
    }
  }
}
```

```

    h2 {
      margin: 0;
      text-align: left !important;
      font-size: 24px;
      font-weight: 700;
      color: var(--ion-color-primary-shade, #38803b);
    }
  }

  ion-card {
    margin-inline: 0px;

    img {
      width: 100%;
      display: block;
      border-radius: 12px 12px 0 0;
    }

    h2 {
      color: var(--ion-color-primary-shade, #38803b);
      margin-top: 16px;
      margin-bottom: 8px;
    }
  }
}

```

page1.page.spec.ts

```

import { ComponentFixture, TestBed } from '@angular/core/testing';
import { Page1Page } from './page1.page';

describe('Page1Page', () => {
  let component: Page1Page;
  let fixture: ComponentFixture<Page1Page>;

  beforeEach(() => {
    fixture = TestBed.createComponent(Page1Page);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });
});

```

page1.page.ts

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-page1',
  templateUrl: './page1.page.html',
  styleUrls: ['./page1.page.scss'],
  standalone: false,
})
export class Page1Page implements OnInit {

  constructor() { }

  ngOnInit() {

  }
}
```

9.3. PAGE2

page2-routing.module.ts

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { Page2Page } from './page2.page';

const routes: Routes = [
  {
    path: '',
    component: Page2Page
  }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class Page2PageRoutingModule {}
```

page2.module.ts

```
import { NgModule, CUSTOM_ELEMENTS_SCHEMA } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { IonicModule } from '@ionic/angular';
import { Page2PageRoutingModule } from './page2-routing.module';
import { Page2Page } from './page2.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    Page2PageRoutingModule
  ],
  declarations: [Page2Page],
  schemas: [CUSTOM_ELEMENTS_SCHEMA],
})
export class Page2PageModule {}
```


page2.page.html

```
<ion-content [fullscreen]="true" class="ion-padding native-scroll"
[scrollY]="true">

  <div class="content">

    <div class="page-hero">
      
      <h2>Welkom op Page 2</h2>
    </div>

    <p>Content </p>

    <div class="matrix-grid">
    </div>

  </div>
</ion-content>
```

page2.page.scss

```
.content {
  max-width: 100%;
  width: 95%;
  margin: 0 auto;

  padding-top: 16px;
  padding-bottom: 16px;

  p {
    color: #555555;
    margin-bottom: 24px;
    font-size: 0.95rem;
    opacity: 0.9;
  }

  .page-hero {
    display: flex;
    flex-direction: column;
    align-items: flex-start;
    width: 100%;
    margin-bottom: 16px;

    .hero-image {
      width: 100%;
      max-height: 250px;
      object-fit: cover;
      border-radius: 8px;
      margin-bottom: 12px;
    }
  }
}
```

```

    h2 {
      margin: 0;
      text-align: left !important;
      font-size: 24px;
      font-weight: 700;
      color: var(--ion-color-primary-shade, #38803b);
    }
  }

  ion-card {
    margin-inline: 0px;

    img {
      width: 100%;
      display: block;
      border-radius: 12px 12px 0 0;
    }

    h2 {
      color: var(--ion-color-primary-shade, #38803b);
      margin-top: 16px;
      margin-bottom: 8px;
    }
  }
}

```

page2.page.spec.ts

```

import { ComponentFixture, TestBed } from '@angular/core/testing';
import { Page2Page } from './page2.page';

describe('Page3Page', () => {
  let component: Page2Page;
  let fixture: ComponentFixture<Page2Page>;

  beforeEach(() => {
    fixture = TestBed.createComponent(Page2Page);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });
});

```

page2.page.ts

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-page2',
  templateUrl: './page2.page.html',
  styleUrls: ['./page2.page.scss'],
  standalone: false,
})
export class Page2Page implements OnInit {

  constructor() { }

  ngOnInit() {

  }
}
```

9.4. PAGE3

page3-routing.module.ts

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { Page3Page } from './page3.page';

const routes: Routes = [
  {
    path: '',
    component: Page3Page
  }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class Page3PageRoutingModule {}
```

page3.module.ts

```
import { NgModule, CUSTOM_ELEMENTS_SCHEMA } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { IonicModule } from '@ionic/angular';
import { Page3PageRoutingModule } from './page3-routing.module';
import { Page3Page } from './page3.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    Page3PageRoutingModule
  ],
  declarations: [Page3Page],
  schemas: [CUSTOM_ELEMENTS_SCHEMA],
})
export class Page3PageModule {}
```

page3.page.html

```
<ion-content [fullscreen]="true" class="ion-padding native-scroll"
[scrollY]="true">

  <div class="content">

    <div class="page-hero">
      
      <h2>Welkom op Page 3</h2>
    </div>

    <p>Content </p>

    <div class="matrix-grid">

    </div>

  </div>
</ion-content>
```

page3.page.scss

```
.content {
  max-width: 100%;
  width: 95%;
  margin: 0 auto;

  padding-top: 16px;
  padding-bottom: 16px;

  p {
    color: #555555;
    margin-bottom: 24px;
    font-size: 0.95rem;
    opacity: 0.9;
  }

  .page-hero {
    display: flex;
    flex-direction: column;
    align-items: flex-start;
    width: 100%;
    margin-bottom: 16px;

    .hero-image {
      width: 100%;
      max-height: 250px;
      object-fit: cover;
      border-radius: 8px;
    }
  }
}
```

```

    margin-bottom: 12px;
  }

  h2 {
    margin: 0;
    text-align: left !important;
    font-size: 24px;
    font-weight: 700;
    color: var(--ion-color-primary-shade, #38803b);
  }
}

ion-card {
  margin-inline: 0px;

  img {
    width: 100%;
    display: block;
    border-radius: 12px 12px 0 0;
  }

  h2 {
    color: var(--ion-color-primary-shade, #38803b);
    margin-top: 16px;
    margin-bottom: 8px;
  }
}
}

```

page3.page.spec.ts

```

import { ComponentFixture, TestBed } from '@angular/core/testing';
import { Page3Page } from './page3.page';

describe('Page3Page', () => {
  let component: Page3Page;
  let fixture: ComponentFixture<Page3Page>;

  beforeEach(() => {
    fixture = TestBed.createComponent(Page3Page);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });
});

```

page3.page.ts

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-page3',
  templateUrl: './page3.page.html',
  styleUrls: ['./page3.page.scss'],
  standalone: false,
})
export class Page3Page implements OnInit {

  constructor() { }

  ngOnInit() {

  }
}
```

10. GLOBAL EN VARIABLES SCSS

1. Variables.scss (De Grondstoffen)

Dit bestand is puur een **woordenboek of opslagplaats**. Het doet zelf nog niks op het scherm, maar bewaart de bouwstenen van je huisstijl.

- **Wat staat erin?** Alleen definities van kleuren, lettertypes en marges (zoals `--ion-color-primary: #2e6930;`).
- **De kracht:** Het bevat de logica voor **Light en Dark Mode**. Het switcht de kleurcodes op de achtergrond om op basis van de systeeminstelling van de gebruiker.
- **Geen styling:** Je zult hier nooit echte CSS-regels vinden zoals `width: 100%` of `border-radius: 8px`.

2. Global.scss (De Toepassing)

Dit bestand is **de uitvoerder**. Het pakt de grondstoffen uit `variables.scss` en past ze daadwerkelijk toe op de onderdelen van je app.

- **Wat staat erin?** Echte CSS-styling en technische lay-outfixes die voor *elke* pagina gelden (zoals de `.native-scroll fix`, de `.matrix-grid` of de styling van `ion-card`).
- **De koppeling:** `global.scss` maakt continu gebruik van de variabelen uit `variables.scss`.

global.scss

Dit bestand regelt de **wereldwijde styling, typografie en technische lay-outfixes** voor de gehele applicatie. Waar lokale bestanden (zoals `login.page.scss`) alleen invloed hebben op één enkel scherm, past de code in dit bestand zich toe op **elke component en elke pagina** in jouw intranet. Het zorgt voor een consistent uiterlijk (de groene huisstijl) en lost hardnekkige mobiele scrollfouten op.

De schilder en de stukadoor. Voordat de componenten in beeld komen, laadt de browser deze stylesheets. `variables.scss` bepaalt de basiskleuren (zoals het bosgroen) en controleert of de telefoon op Dark Mode staat. `global.scss` past direct de universele scrolloverrides en lettertypes toe op de lege `index.html`.

```
@import "@ionic/angular/css/core.css";
@import "@ionic/angular/css/normalize.css";
@import "@ionic/angular/css/structure.css";
@import "@ionic/angular/css/typography.css";
@import "@ionic/angular/css/display.css";

@import "@ionic/angular/css/padding.css";
@import "@ionic/angular/css/float-elements.css";
@import "@ionic/angular/css/text-alignment.css";
@import "@ionic/angular/css/text-transformation.css";
@import "@ionic/angular/css/flex-utils.css";
```



```

html, body, ion-app, ion-router-outlet, .ion-page {
  height: 100% !important;
  overflow: hidden !important;
}

html {
  font-size: 1rem;
  line-height: 1.618;
}

body {
  font-family: Georgia, 'Times New Roman', Times, serif;
}

h1, h2, h3, h4, h5, h6, ion-title, ion-label, ion-input {
  font-family: var(--ion-font-family), system-ui, -apple-system, sans-serif !important;
  font-weight: 700;
  letter-spacing: -0.02em;
}

ion-button {
  --border-radius: 8px !important;
  font-weight: 600;
}

ion-card {
  margin: 10px !important;
  display: block;
  border-radius: 12px !important;
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.04) !important;
  background: rgba(255, 255, 255, 0.8) !important;
  -webkit-backdrop-filter: blur(4px) !important;
  backdrop-filter: blur(4px) !important;
}

ion-toolbar {
  --background: transparent;
  --border-color: transparent;
  border-bottom: 1px solid var(--ion-color-step-100);
}

ion-content {
  --padding-top: 13px;
  --padding-bottom: 13px;
  --padding-start: 13px;
  --padding-end: 13px;
  --background: linear-gradient(to bottom, #a2d2a4, #e8f5e9);
}

```

```

ion-tab-bar {
  --background: #e8f5e9;
  border-top: 1px solid rgba(0, 0, 0, 0.05);
}

ion-tab-button {
  --color: #557a57;
  --color-selected: var(--ion-color-primary);
}

ion-content.native-scroll {
  --overflow: scroll !important;
  overflow: visible !important;
  height: 100% !important;
}

ion-content.native-scroll::part(background) {
  background: linear-gradient(to bottom, #a2d2a4, #e8f5e9) !important;
}

ion-content.native-scroll::part(scroll) {
  overflow-y: scroll !important;
  position: relative;
  touch-action: pan-y !important;
  -webkit-overflow-scrolling: touch !important;
}

.matrix-grid {
  display: block !important;
  height: auto !important;
  min-height: max-content !important;
  padding: 0 !important;
  margin: 0 !important;
  width: 100%;

  .grid-row-zero,
  .col-padding-zero {
    padding: 0 !important;
    margin: 0 !important;
  }
}

.matrix-btn {
  margin: 0 !important;
  --border-radius: 0;
  height: 60px !important;
}

```

```
.header-banner {
  border-radius: 8px;
  width: 100%;
  height: 200px;
  object-fit: cover;
  display: block;
}

.card-banner {
  width: 100%;
  height: 120px;
  object-fit: cover;
}

.secure-note {
  margin-top: 16px;
  opacity: 0.7;
}

.full-width-form {
  width: 100%;
}

.tab-nav-btn.active-tab {
  --color: var(--ion-color-primary) !important;
}

.bg-light-blue {
  background-color: #add8e6 !important;
}
```

theme\variables.scss

Dit bestand regelt de **visuele identiteit en het kleurenpalet** van je complete intranet. In plaats van dat je kleuren hard codeert (zoals #2e6930), definieer je hier CSS-variabelen. Wijzig je hier één kleurcode, dan verandert die kleur direct in de hele app. Daarnaast zorgt dit bestand voor de automatische omschakeling tussen **Light Mode** en **Dark Mode**.

```
:root {  
  /** Primary Color (Light Mode - Bosgroen) */  
  --ion-color-primary: #2e6930;  
  --ion-color-primary-rgb: 46, 105, 48;  
  --ion-color-primary-contrast: #ffffff;  
  --ion-color-primary-contrast-rgb: 255, 255, 255;  
  --ion-color-primary-shade: #295d2a;  
  --ion-color-primary-tint: #437845;  
  
  /** Secondary Color (Light Mode - Saliegroen) */  
  --ion-color-secondary: #558b2f;  
  --ion-color-secondary-rgb: 85, 139, 47;  
  --ion-color-secondary-contrast: #ffffff;  
  --ion-color-secondary-contrast-rgb: 255, 255, 255;  
  --ion-color-secondary-shade: #4b7a2a;  
  --ion-color-secondary-tint: #669643;  
  
  /** Passende Achtergrond & Tekst (Light Mode) */  
  --ion-background-color: #fcfbfa;  
  --ion-background-color-rgb: 252, 251, 250;  
  --ion-text-color: #1c1b1a;  
  --ion-text-color-rgb: 28, 27, 26;  
  
  /** Subtiële olijf/grijstinten (stappen van 50 tot 900) */  
  --ion-color-step-50: #f6f7f3;  
  --ion-color-step-100: #edf0e6;  
  --ion-color-step-200: #dee3d3;  
  --ion-color-step-300: #ced6bf;  
  --ion-color-step-500: #afba99;  
  --ion-color-step-700: #717d5c;  
  --ion-color-step-900: #353b2a;  
  
  /** Global Typography */  
  --ion-font-family: 'Roboto', sans-serif;  
  --line-height: 1.618;  
  --font-size: 1rem;  
}
```

```

/* Automatische Dark Mode instellingen */
@media (prefers-color-scheme: dark) {
  :root {
    /** Primary Color (Dark Mode - Iets lichter groen voor leesbaarheid) */
    --ion-color-primary: #4e9e51;
    --ion-color-primary-rgb: 78, 158, 81;
    --ion-color-primary-contrast: #ffffff;
    --ion-color-primary-contrast-rgb: 255, 255, 255;
    --ion-color-primary-shade: #458b47;
    --ion-color-primary-tint: #60a863;

    /** Secondary Color (Dark Mode) */
    --ion-color-secondary: #7cb342;
    --ion-color-secondary-rgb: 124, 179, 66;
    --ion-color-secondary-contrast: #ffffff;
    --ion-color-secondary-contrast-rgb: 255, 255, 255;
    --ion-color-secondary-shade: #6d9e3b;
    --ion-color-secondary-tint: #89ba55;

    /** Passende Donkere Achtergrond & Tekst (Dark Mode) */
    --ion-background-color: #111411; /* Heel donkergroen/zwart */
    --ion-background-color-rgb: 17, 20, 17;
    --ion-text-color: #e3e6e3;
    --ion-text-color-rgb: 227, 230, 227;

    /** Donkere stappen voor kaarten en lijnen */
    --ion-color-step-50: #191c19;
    --ion-color-step-100: #242924;
    --ion-color-step-200: #303730;
    --ion-color-step-300: #3c453c;
    --ion-color-step-500: #697869;
    --ion-color-step-700: #9cb09c;
    --ion-color-step-900: #cfdbcf;
  }
}

/** Algemene Component Styling */
.ion-padding-top {
  --padding-top: 25px;
}

ion-item {
  --background: transparent;
  --color: var(--ion-text-color);
}

ion-toolbar {
  --background: transparent;
}

```

11. INDEX.HTML

Dit bestand is het **absolute startpunt** van je app in de browser. Het laadt de configuraties voor het mobiele scherm (viewport) en bereidt de Light/Dark mode voor. De vernieuwde CSP-beveiligingstag opent nu specifiek de poorten naar googleapis.com en firebaseapp.com bij connect-src. Hierdoor blokkeert de browser de dataoverdracht van jouw LoginService naar de cloud niet meer.

```
<!DOCTYPE html>
<html lang="nl">

<head>
  <meta charset="utf-8" />
  <title>Ionic App</title>

  <base href="/" />

  <meta name="color-scheme" content="light dark" />
  <meta name="viewport" content="viewport-fit=cover, width=device-width,
initial-scale=1.0, minimum-scale=1.0, maximum-scale=1.0, user-scalable=no" />
  <meta name="format-detection" content="telephone=no" />
  <meta name="msapplication-tap-highlight" content="no" />

  <!-- Gecorrigeerde CSP die zowel DevTools, Pixabay als Firebase Cloud
toestaat -->
  <meta http-equiv="Content-Security-Policy" content="
    default-src 'self' data: gap: https://*.gstatic.com
https://*.firebaseio.com http://localhost:* ws://localhost:*;
    style-src 'self' 'unsafe-inline';
    script-src 'self' 'unsafe-inline' 'unsafe-eval';
    img-src 'self' data: https://pixabay.com https://*.pixabay.com;
    connect-src 'self' https://*.googleapis.com https://*.firebaseapp.com
http://localhost:* ws://localhost:* http://127.0.0.1:* ws://127.0.0.1:*;
  ">

  <link rel="icon" type="image/png" href="assets/icon/favicon.png" />

  <meta name="mobile-web-app-capable" content="yes" />
  <meta name="apple-mobile-web-app-status-bar-style" content="black" />
</head>

<body>
  <app-root></app-root>
</body>

</html>
```

12. MAIN.TS

Dit bestand is jouw **main.ts** (de startmotor of de ontsteking van de gehele applicatie).

Wat is de functie van dit specifieke bestand?

De enige en cruciale taak van main.ts is het **in gang zetten van het opstartproces (bootstrapping)** van je app. Zodra index.html in de browser geladen is, roept deze de gecompileerde versie van main.ts aan.

Dit is wat de code in dit bestand precies doet:

- **platformBrowserDynamic():**
Dit is de methode van Angular die de app voorbereidt om dynamisch in een standaard webversie (de browser) te draaien. Het zorgt ervoor dat de code direct ter plekke in de browser vertaald en uitgevoerd kan worden (JIT-compilatie of *Just-In-Time*).
- **.bootstrapModule(AppModule):**
Dit is de daadwerkelijke startknop. Het vertelt Angular: *"Pak de hoofdgereedschapskist (AppModule), start de applicatiemotor op en begin met het tekenen van de hoofdcomponent (AppComponent) op de plek van de <app-root> tag."*
- **.catch(err => console.log(err)):**
Mocht er tijdens het allereerste opstarten van de app een fatale fout optreden (bijvoorbeeld door een kapotte import of een syntaxfout in je AppModule), dan vangt deze regel de fout netjes op en print de technische melding in de Developer Console van je browser. Dit voorkomt dat de app geruisloos blijft hangen op een wit scherm zonder dat je weet waarom.

```
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';

import { AppModule } from './app/app.module';

platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.log(err));
```

13. Firebase authentication en databases

De enige en cruciale taak van **environment.ts** is het leveren van de configuratiedata (de API-sleutels) waarmee de frontend (je Ionic-app) verbinding maakt met de backend-services (Firebase). Het fungeert als de brug/adreskaart, zodat Google exact weet naar welke database en authenticatie-omgeving de gegevens gestuurd moeten worden.

<https://firebase.google.com>

Maak een project en database aan

Authenticatieproces -> sign-in method Email/password Enable Save

General -> API-key (plak deze in VS in environment.ts)

Environment.ts (API-key verbindt met Firebase)

```
export const environment = {  
  production: false,  
  firebaseConfig: {  
    apiKey: "AIzaSyAQTM1b_fAPQLtam7sp5TKwZIx7n-oFgy4",  
    authDomain: "ionic-angular-oefenapp.firebaseio.com",  
    projectId: "ionic-angular-oefenapp",  
    storageBucket: "ionic-angular-oefenapp.firebaseio.com",  
    messagingSenderId: "61828244565",  
    appId: "1:61828244565:web:5e055f799b4a9edaa6a0dc"  
  }  
};
```

13.1. Live Google Firebase backend

Wanneer je een bestaand Angular/Ionic project ombouwt van een lokale simulatie (zoals met localStorage) naar een live **Google Firebase** backend (Auth & Firestore), moet je specifiek de volgende **7 bestanden** aanpassen of aanmaken.

Hier is het exacte overzicht van de bestanden en wat erin moet veranderen:

1. src/environments/environment.ts (en environment.prod.ts)

- **Wat moet hierin?:** De geheime sleutels en configuratiedata van jouw Firebase-project.
- **Aanpassing:** Je plakt hier het configuratie-object dat je van de Firebase Console krijgt (met de apiKey, authDomain, projectId, etc.).

2. src/app/app.module.ts (of app.config.ts bij standalone)

- **Wat moet hierin?:** De initialisatie van Firebase binnen de app-motor.
- **Aanpassing:** Je importeert hier de Firebase-modules en linkt ze aan de sleutels uit je environment.ts bestand via:
typescript
provideFirebaseApp(() => initializeApp(environment.firebase)),
provideAuth(() => getAuth()),
provideFirestore(() => getFirestore())
Wees voorzichtig met code.

3. src/app/services/login.service.ts

- **Wat moet hierin?:** De daadwerkelijke communicatielogica met de cloud.
- **Aanpassing:** Je verwijdert de oude localStorage code. In plaats daarvan injecteer je Auth en gebruik je officiële Firebase-functies zoals signInWithEmailAndPassword, createUserWithEmailAndPassword, signOut en sendPasswordResetEmail.

4. src/app/login/login.page.ts

- **Wat moet hierin?:** De foutafhandeling op het loginscherm.
- **Aanpassing:** Omdat Firebase eigen specifieke foutcodes teruggeeft, pas je de catch-blokken in de onSubmit() en onForgotPassword() methoden aan om te controleren op Firebase-fouten zoals 'auth/invalid-credential' of 'auth/user-not-found'.

5. src/app/signup/signup.page.ts

- **Wat moet hierin?:** De 2-staps backend brug (Registratie + Database).
- **Aanpassing:** Naast het aanmaken van de gebruiker in Firebase Auth, voeg je hier de logica toe om met setDoc of addDoc direct een nieuw document aan te maken in de Cloud Firestore users collectie onder het unieke user.uid.

6. src/app/guards/login.guard.ts

- **Wat moet hierin?:** De uitsmijter die de status controleert.
- **Aanpassing:** Je past de guard aan zodat deze luistert naar de live RxJS usersIsAuthenticated\$ stream van de LoginService (die gekoppeld is aan de Firebase user(auth) observer), in plaats van een harde check op een lokale browser-variabele.

7. src/app/homepage/homepage.page.ts (en overige contentpagina's)

- **Wat moet hierin?:** De uitlogknop en het ophalen van gebruikersgegevens.
 - **Aanpassing:** De onLogout() methode moet nu asynchroon de logout() van de service afwachten. Daarnaast pas je eventuele componenten aan die de naam van de gebruiker tonen; deze moeten nu via een Firestore-query live de voornaam ophalen op basis van het ingelogde UID.
-